

GATE CSE · COMPLETE GUIDE

# Computer Networks

## Theory · Shortcuts · Solved Questions

The beginner-friendly route to full marks

Layers · Subnetting · Routing · TCP · Congestion Control · DNS

# Computer Networks

for GATE CSE

The complete beginner-friendly guide
Theory · Shortcuts · Solved Questions

# About this book

**Who this book is for.** Someone opening Computer Networks (CN) for the *first time* who wants to score full marks in GATE without sitting through long lectures.

**How it was built.** (1) Your handwritten lecture notes — every topic and every example in them is included; (2) your PYQ analysis (1987–2026) — topics that appear *every year* get the most space, extra worked examples and exam shortcuts.

## How to read the boxes

FROM YOUR NOTES An example or fact taken directly from **your lecture notes**, solved step by step.

EXAM SHORTCUT A **trick** to save time in the exam.

TRAP A mistake examiners count on. Read these twice.

GATE-STYLE QUESTION A question written in the style of the PYQs, with a full solution.

INTUITION A plain-English explanation or analogy.

KEY IDEA A definition or rule you must remember.

KEY FORMULA A formula worth memorising, typeset for quick recall.

## 4pt Exam-priority tiers

★★★ **Tier 1 – sure-shot.** Asked in $\geq 8$ of the last 9 GATE papers. Master completely.

★★ **Tier 2 – high probability** (6–7 of the last 9 papers).

★ **Tier 3 – lumpy.** Big marks when it comes, sometimes absent.

☐ **Tier 4 – low yield.** Skim last.

# Contents


0 Priority Map . . . . . 1
0.1 How many marks does CN carry? . . . . . 1
0.2 Topic clusters in CN — ranked by importance . . . . . 1
0.3 Year-by-year heat-map (marks from each cluster) . . . . . 2
0.4 What exactly does GATE ask? (question styles 2018–26) . . . . . 3
0.5 Suggested study order (by return on time) . . . . . 3

1 Foundations: Layers, Protocols and Devices . . . . . 5
1.1 What is a computer network? . . . . . 5
1.2 Protocol . . . . . 6
1.3 The 5-layer TCP/IP stack . . . . . 6
1.4 Encapsulation (how data travels down the stack) . . . . . 7
1.5 Who operates at which layer? . . . . . 8
1.6 “Which layer is responsible for …?” (1-mark favourites) . . . . . 8

2 Performance: Bandwidth, Throughput, Delays . . . . . 9
2.1 Bandwidth vs throughput . . . . . 9
2.2 Latency (total delay) . . . . . 10
2.3 Store-and-forward & multi-hop delay . . . . . 11
2.4 Bandwidth-Delay Product (BDP) . . . . . 11
2.5 Link vs channel . . . . . 11
2.6 Quick exam checklist . . . . . 12

3 Data Link Layer (DLL) — Role and Framing . . . . . 13
3.1 What does the Data Link Layer do? . . . . . 13
3.2 Framing . . . . . 14

**4 Error Control . . . 17**
4.1 Types of errors . . . 17
4.2 Dataword → Codeword (block coding) . . . 18
4.3 Hamming distance . . . 18
4.4 Linear block codes . . . 19
4.5 Parity check code . . . 19
4.6 Cyclic codes and CRC ★ . . . 19
4.7 Hamming code (single-error correction) □ (2021) . . . 22
4.8 Internet checksum (used in IP, TCP, UDP) (see IP header) . . . 23

**5 Flow Control and Sliding Window . . . 24**
5.1 Why flow control? . . . 24
5.2 Stop-and-Wait (SW) . . . 24
5.3 Why Sliding Window? . . . 26
5.4 Go-Back-N (GBN) . . . 27
5.5 Selective Repeat (SR) . . . 28
5.6 Efficiency and throughput of sliding window . . . 29

**6 MAC Layer, Ethernet and Devices . . . 31**
6.1 The multiple-access problem . . . 31
6.2 ALOHA ★ (formulas are favourites) . . . 32
6.3 CSMA — Carrier Sense Multiple Access . . . 33
6.4 CSMA/CD — Collision Detection (Ethernet) ★ . . . 33
6.5 Binary Exponential Backoff algorithm ★ . . . 34
6.6 CSMA/CA (Wi-Fi) — bonus □ . . . 35
6.7 Ethernet frame (IEEE 802.3) ★ . . . 36
6.8 Controlled access and channelization (theory, 1-markers) □ . . . 36
6.9 Hubs, bridges, switches, routers — domains ★ . . . 36

**7 IP Addressing, Classes, Subnetting, VLSM . . . 38**
7.1 IPv4 address basics . . . 39
7.2 Classful addressing . . . 39
7.3 Classless addressing (CIDR notation) . . . 41
7.4 Subnetting . . . 42
7.5 VLSM — Variable-Length Subnet Masks ★★★ . . . 44

**8 Forwarding, Longest-Prefix Match, CIDR and Supernetting . . . 47**
8.1 The routing (forwarding) table . . . 47
8.2 Forwarding algorithm ★★★ . . . 47

8.3 Supernetting / route aggregation (CIDR) ★ . . . 49
8.4 CIDR block allocation by an ISP ★ (GATE style) . . . 51
8.5 NAT (bonus, for “IP-header facts” 1-markers) □ . . . 52

**9 IP Header, Fragmentation, ICMP and ARP . . . 53**
9.1 The IPv4 header . . . 53
9.2 MTU, MSS and why fragmentation exists ★ . . . 55
9.3 Fragmentation ★★ . . . 56
9.4 ICMP — Internet Control Message Protocol ★ . . . 59
9.5 ARP — Address Resolution Protocol ★ . . . 60

**10 Routing: Distance Vector, Link State and Real Protocols . . . 63**
10.1 Routing vs forwarding . . . 63
10.2 Distance Vector Routing (DVR) ★★ . . . 64
10.3 Count-to-Infinity problem ★★ . . . 66
10.4 Link State Routing (LSR) ★ *(notes: “Pending” — completed here)* . . . 67
10.5 Real routing protocols ★ (1-mark matching) . . . 68

**11 Transport Layer — Ports, UDP and TCP Basics . . . 69**
11.1 What does the transport layer do? . . . 69
11.2 UDP — User Datagram Protocol ★ . . . 70
11.3 TCP — Transmission Control Protocol ★★★ . . . 71
11.4 Sequence numbers and acknowledgement numbers ★★★ . . . 72
11.5 Windows in TCP . . . 73

**12 TCP Connection, Acknowledgement, Retransmission, Flow Control, Timers . . . 75**
12.1 Connection establishment — 3-way handshake ★★★ . . . 75
12.2 Connection termination ★★ . . . 77
12.3 TCP state machine ★ . . . 78
12.4 Acknowledgement policy ★ . . . 79
12.5 Retransmission ★★ . . . 80
12.6 TCP timers (4 important) . . . 81
12.7 Flow control in TCP ★★ . . . 81
12.8 Sequence-number wrap-around ★ (asked in 2018) . . . 82

**13 TCP Congestion Control . . . 83**
13.1 What is congestion? . . . 83
13.2 The congestion window (cwnd) . . . 84

13.3 Phase 1 — Slow Start . . . . . 84
13.4 Phase 2 — Congestion Avoidance . . . . . 85
13.5 Reacting to congestion — Tahoe and Reno ★★★ . . . . . 85
13.6 TCP vs UDP (1-mark favourite) . . . . . 88

**14 Application Layer . . . . . 89**
14.1 Client–server idea . . . . . 89
14.2 E-mail system ★ . . . . . 90
14.3 DNS — Domain Name System ★★ . . . . . 91
14.4 HTTP — HyperText Transfer Protocol ★★ . . . . . 94
14.5 FTP — File Transfer Protocol ★ . . . . . 95
14.6 DHCP — Dynamic Host Configuration Protocol ★ . . . . . 95
14.7 Other protocols — quick facts . . . . . 96
14.8 ★★ MASTER TABLE — Protocols, ports, transport (memorise!) . . . . 97

**15 Practice Sets with Full Solutions . . . . . 98**
Set A — IP addressing & subnetting (★★★ do all) . . . . . 98
Set B — TCP, flow & congestion control (★★★ do all) . . . . . 100
Set C — Fragmentation, ICMP, ARP . . . . . 103
Set D — Routing . . . . . 104
Set E — Application layer . . . . . 104
Set F — MAC, Ethernet, devices . . . . . 105
Set G — Error control & framing . . . . . 105
Set H — Delays . . . . . 106

**16 Formula Sheet & 30-Minute Revision . . . . . 107**
16.1 One-page formulas . . . . . 107
16.2 Question type → method . . . . . 109
16.3 Top exam traps . . . . . 111
16.4 30-minute last-day revision plan . . . . . 111

**17 Appendices . . . . . 112**
Appendix A — Where each item of your lecture notes is covered . . . . . 112
Appendix B — Corrections to the handwritten notes (so you don't lose marks) 114
Appendix C — How to attempt PYQs after this book . . . . . 115

CHAPTER

# 0

# Priority Map

*(from your PYQ analysis)*

## 0.1 How many marks does CN carry?

| Item | Value |
|---|---|
| Average marks per paper (2018–26) | **7.8 / 100** |
| Range seen | 6 – 10 |
| Last 5 papers average (2022–26) | 8.5 |
| GATE 2026 | 8.5 |
| Typical split | ~ 5 one-mark + ~ 5–6 two-mark questions |

## 0.2 Topic clusters in CN — ranked by importance

| Rank | Cluster | Tier | Avg marks/paper | Years asked ('18–'26) | In '24 / '25 / '26 |
|---|---|---|---|---|---|
| 1 | **IP addressing, subnetting & CIDR** | ★★★ | 1.67 | 8 / 9 | ✔ ✔ ✔ |
| 2 | **TCP, UDP & congestion control** | ★★★ | 1.56 | 8 / 9 | ✔ ✔ ✔ |

| Rank | Cluster | Tier | Avg marks/paper | Years asked ('18–'26) | In '24 / '25 / '26 |
|---|---|---|---|---|---|
| 3 | **Application-layer protocols & OSI/TCP-IP model** | ★★ | 1.28 | 7 / 9 | ✔ ✔ ✔ |
| 4 | **Routing (distance-vector, link-state)** | ★★ | 0.89 | 7 / 9 | ✔ ✔ ✔ |
| 5 | MAC, LAN & Ethernet (CSMA/CD, Aloha, bridges) | ★ | 0.83 | 5 / 9 | ✔ · · |
| 6 | IP packet, fragmentation, ARP/ICMP | ★ | 0.72 | 4 / 9 | ✔ ✔ ✔ |
| 7 | Flow control (stop-and-wait, sliding window) | ★ | 0.44 | 4 / 9 | · ✔ ✔ |
| 8 | Error detection & coding (CRC, Hamming, bit-stuffing) | ☐ | 0.33 | 2 / 9 | · · ✔ |
| 9 | Network flow & misc. | ☐ | 0.11 | 1 / 9 | · ✔ · |

## 0.3 Year-by-year heat-map (marks from each cluster)

| Cluster | '18 | '19 | '20 | '21 | '22 | '23 | '24 | '25 | '26 |
|---|---|---|---|---|---|---|---|---|---|
| IP addressing, subnetting, CIDR | 2 | 2 | 2 | – | 2 | **4** | 1 | 1.5 | 0.5 |
| TCP, UDP, congestion control | 1 | – | 2 | 2.5 | 2 | 2 | 1.5 | 0.5 | **2.5** |
| Application layer & OSI | – | **3** | 1 | 1.5 | **3** | – | 0.5 | 0.5 | 2 |
| Routing | – | – | 1 | 1 | 2 | 1 | 2 | 0.5 | 0.5 |
| MAC / LAN / Ethernet | 2 | 2 | – | 1 | 1 | – | 1.5 | – | – |
| IP packet, fragmentation, ARP/ICMP | 1 | – | – | – | – | – | **2.5** | 2 | 1 |
| Flow control | 1 | – | – | – | – | 1 | – | 1 | 1 |
| Error detection & coding | – | – | – | 2 | – | – | – | – | 1 |
| **CN total** | 7 | 7 | 6 | 8 | 10 | 8 | 9 | 7 | 8.5 |

EXAM SHORTCUT

**Reading the table:** Subnetting and TCP have **never both been missing** in the same year. Fragmentation/ARP/ICMP became a recurring theme only recently (2024–26) — don't skip it.

## 0.4 What exactly does GATE ask? (question styles 2018–26)

| Topic | Question styles that actually appeared |
|---|---|
| IP addressing / subnetting | Subnet mask, number of hosts, prefix range, CIDR block allocation • Forwarding table with longest-prefix match (LPM) / next hop • IP-header facts, DNS + IP interplay |
| TCP | Connection setup/teardown: 3-way handshake, sequence numbers, MSS/MTU, timers • **Congestion window after k RTTs**, slow start/AIMD, throughput |
| Application layer | Protocol roles (DNS, HTTP, SMTP, POP, ARP, ICMP, OSI-layer matching) • Link calculations (sliding-window throughput, satellite propagation delay) |
| Routing | Forwarding + end-to-end transfer-time calculation • RIP/OSPF/BGP statements • Distance-vector table updates, **count-to-infinity** |
| MAC / LAN | CSMA/CD, pure ALOHA efficiency • Switches / Ethernet segments / LAN design |
| IP packet | Header fields modified by routers (TTL, checksum), ARP facts, CIDR block assignment • **Fragmentation**: number of fragments, offset, MTU |
| Flow control | Stop-and-wait channel utilisation • Sliding window • **Wrap-around time** of sequence numbers |
| Error detection | CRC polynomial properties • Hamming code |

## 0.5 Suggested study order (by return on time)

| Order | Chapter | Why |
|---|---|---|
| 1 | Ch 7 + 8 (IP addressing, subnetting, CIDR, forwarding) | highest marks, pure calculation, can be mastered with 3 tricks |
| 2 | Ch 11 + 12 + 13 (UDP, TCP, congestion control) | 2nd-highest, very predictable questions |
| 3 | Ch 14 (Application layer) | easy 1-markers: just learn the facts |
| 4 | Ch 10 (Routing) | DV tables + count-to-infinity = 2 marks nearly every year |
| 5 | Ch 9 (Fragmentation, ARP, ICMP, header) | frequent since 2024 |
| 6 | Ch 6 (MAC / Ethernet) | CSMA/CD + Aloha formulas |

| Order | Chapter | Why |
|---|---|---|
| 7 | Ch 5 (Flow control) | a few formulas |
| 8 | Ch 1, 2, 3, 4 | foundations (read once; CRC/Hamming for 1-mark questions) |

**TRAP**

GATE has no fixed quota per topic. "Sure-shot" = historically near-certain. Treat tiers as *odds*, not guarantees.

CHAPTER

# 1

# Foundations: Layers, Protocols and Devices

*(from absolute zero)*

IN THIS CHAPTER


- 1.1 What is a computer network?
- 1.2 Protocol
- 1.3 The 5-layer TCP/IP stack
- 1.4 Encapsulation (how data travels down the stack)
- 1.5 Who operates at which layer?
- 1.6 "Which layer is responsible for …?" (1-mark favourites)


## 1.1 What is a computer network?

A **network** is a set of devices (**hosts** such as laptops, phones, servers) connected by **links** (cables, wireless) and **routers/switches**, so they can exchange data.

INTUITION

**Analogy – the postal system.** You write a letter (data). It is put in an envelope with addresses (headers). Local post office → city sorting centre → destination city → postman → your friend. Different people handle different jobs. Networking does the same, split into **layers**.

## 1.2 Protocol

KEY IDEA

**Protocol** = *a set of rules agreed upon and followed by two or more devices* (what to send, in what format, in what order, what to do on errors).

Example: before talking on a phone, both say "hello". That agreed rule is a protocol.

## 1.3 The 5-layer TCP/IP stack

We study networking as a **stack of 5 layers**. (*your notes: Layer 5 → Layer 1*)

| # | Layer | One-line job | PDU (data unit name) | Address used | Example protocols | Device |
|---|---|---|---|---|---|---|
| 5 | **Applicatior** | Services directly used by programs | Message | URL / e-mail / host-name | HTTP, FTP, SMTP, DNS, DHCP, POP3, IMAP | – |
| 4 | **Transport** | *Process-to-process* delivery, reliability, flow & congestion control | Segment (TCP) / Datagram (UDP) | **Port number** (16 bit) | TCP, UDP | – |
| 3 | **Network** | *Host-to-host* delivery across many networks; routing | Packet / Datagram | **IP address** (32 bit) | IP, ICMP, ARP*, RIP/OSPF/BGP | **Router** |
| 2 | **Data Link** | *Hop-to-hop* (*node-to-node*) delivery on one link | **Frame** | **MAC address** (48 bit) | Ethernet, PPP, Wi-Fi | **Switch, Bridge**, NIC |
| 1 | **Physical** | Moves raw **bits** as signals | Bit | – | cables, fibre, signals | **Hub, Repeater** |

*ARP helps the network layer but its packet sits directly inside a data-link frame.

EXAM SHORTCUT

**Memory trick (top → bottom): A**ll **T**eachers **N**eed **D**ata **P**lanning → **A**pplication, **T**ransport, **N**etwork, **D**ata-link, **P**hysical.

KEY IDEA

**Rule:** *Each layer provides service to the layer above it and takes service from the layer below it.*

### OSI model (7 layers) — only for matching questions

| OSI # | OSI layer | Maps to TCP/IP |
|---|---|---|
| 7 | Application | Application |
| 6 | Presentation (encryption, compression, format) | **Application** |
| 5 | Session (dialog control, sync points) | Application |
| 4 | Transport | Transport |
| 3 | Network | Network |
| 2 | Data link | Data link |
| 1 | Physical | Physical |

## 1.4 Encapsulation (how data travels down the stack)

Each layer adds its own **header** (data-link also adds a **trailer**) — like putting a letter in an envelope, then in a bigger envelope.

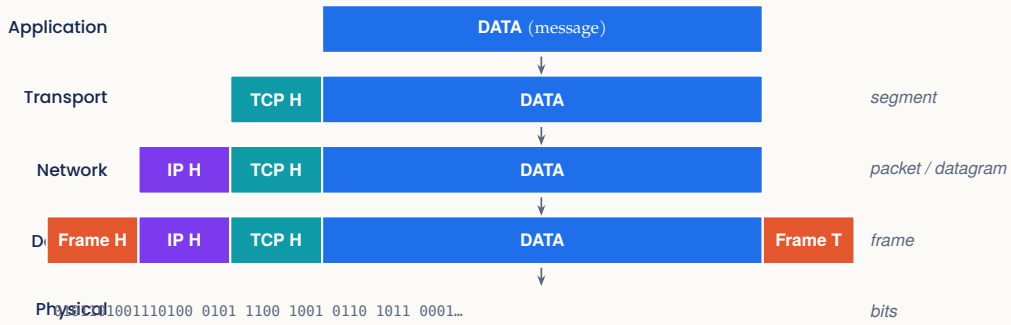



**Figure 1.1** Encapsulation: each layer adds its own header; the data-link layer adds a trailer too.

At the receiver the headers are removed layer by layer (**decapsulation**).

## 1.5 Who operates at which layer?

- **End hosts** (mobiles, laptops, servers) work on **all five** layers.
- **Routers** work only up to **Layer 3** (network layer).
- **Switches / bridges** work up to **Layer 2**.
- **Hubs / repeaters** work at **Layer 1** only.
- Between the *same layer of two different stacks* there is a **logical (virtual) connection** — transport talks to transport, network to network, etc., even though the actual bytes physically go down and up the stack.

## 1.6 "Which layer is responsible for …?" (1-mark favourites)

| Responsibility | Layer | Scope |
|---|---|---|
| Framing, MAC addressing, flow/error control on a link | Data link | **hop-to-hop** |
| Logical (IP) addressing, routing, fragmentation | Network | **host-to-host** |
| Port addressing, reliability, flow control, congestion control, segmentation | Transport | **process-to-process (end-to-end)** |
| DNS, HTTP, SMTP … | Application | user services |

TRAP

*Reliability end-to-end is done by the transport layer* (*TCP*), not by IP. **IP is unreliable** (best-effort).

CHAPTER

# 2

# Performance: Bandwidth, Throughput, Delays

IN THIS CHAPTER


- 2.1 Bandwidth vs throughput
- 2.2 Latency (total delay)
- 2.3 Store-and-forward & multi-hop delay
- 2.4 Bandwidth-Delay Product (BDP)
- 2.5 Link vs channel
- 2.6 Quick exam checklist


## 2.1 Bandwidth vs throughput

- **Bandwidth (BW)** of a channel = amount of data that **can** be sent per unit time (bits/s, bytes/µs, ...). It is the *ideal* speed.
- **Throughput** = amount of data **actually** transferred per unit time. It is the *actual* speed.

FROM YOUR NOTES

**Example (notes).** A network with bandwidth 10 Mbps can pass only an average of 12,000 frames per minute, each carrying $10^4$ bits. Throughput? - Bits per minute = $12{,}000 \times 10^4 = 12 \times 10^7$ bits/min - Per second = $12 \times 10^7 / 60 = \mathbf{2 \times 10^6}$ **bps = 2 Mbps**

### Unit conventions (GATE standard)

| Quantity | Convention |
|---|---|
| **Data rates** (bps, Kbps, Mbps, Gbps) | decimal: 1 K = $10^3$, 1 M = $10^6$, 1 G = $10^9$ |
| **Sizes** (KB, MB) | binary: 1 KB = $2^{10}$ B, 1 MB = $2^{20}$ B (unless the question says otherwise) |
| Bits vs Bytes | 1 B = 8 b. Convert before dividing! |

## 2.2 Latency (total delay)

LATENCY

$$\text{Latency} = \underbrace{T_p}_{\text{propagation}} + \underbrace{T_t}_{\text{transmission}} + \underbrace{\text{queuing}}_{\text{delay}} + \underbrace{\text{processing}}_{\text{delay}}$$

| Delay | Meaning | Formula |
|---|---|---|
| **Transmission time** ($T_t$) | time for the sender to **push all bits onto the link** | $T_t = \frac{L}{B}$ (L = message size in bits, B = bandwidth) |
| **Propagation time** ($T_p$) | time for **one bit** to travel from A to B | $T_p = \frac{d}{v}$ (d = distance, v = propagation speed) |
| **Queuing delay** | time a packet waits in a router's queue | given |
| **Processing delay** | time router takes to examine header, decide route | given |

INTUITION

**Analogy:** Cars leaving a toll gate. *Transmission time* = time for all cars of a convoy to pass the gate. *Propagation time* = time one car takes to drive down the highway.

EXAM SHORTCUT

**Typical speeds:** copper $\approx 2\times10^8$ m/s; fibre $\approx 2\times10^8$ m/s; light/radio in vacuum $3\times10^8$ m/s.

### Example 2.1

A 1000-byte packet is sent on a 10 Mbps link of length 2000 km, v = $2\times10^8$ m/s. Find the time until the last bit arrives. - $T_t$ = 8000 bits / $10^7$ = **0.8 ms** - $T_p$ = $2\times10^6$ m / $2\times10^8$ = **10 ms** - Total = **10.8 ms**

## 2.3 Store-and-forward & multi-hop delay

A router receives a **whole packet** before forwarding it (store-and-forward). - For **one** packet over **h links** (equal rate, $T_p$ ignored): time = **h** $\times$ $T_t$. - For a message split into **n packets** over h links (pipelining): time = **(h + n − 1)** $\times$ $T_t$ ($T_t$ is per packet).

**Example:** message 10,000 bits, 3 links of 1 Mbps, ignore $T_p$. - No splitting: 3 $\times$ 10 ms = **30 ms**. - Split into 10 packets of 1000 bits ($T_t$ = 1 ms): (3 + 10 − 1) $\times$ 1 ms = **12 ms**. ✔ Splitting is faster.

## 2.4 Bandwidth-Delay Product (BDP)

KEY FORMULA

$$\text{BDP} = B \times \text{RTT} \qquad (\text{RTT} = 2T_p)$$

amount of data that fits "in the pipe" in one round trip.

**Example:** 100 Mbps link, RTT 40 ms $\rightarrow$ BDP = $10^8 \times 0.04 = 4\times10^6$ bits = **500,000 bytes**. To keep the link fully busy the sender needs a window $\geq$ BDP.

## 2.5 Link vs channel

A **link** is a *single physical connection*. On one link **several channels** can exist, separated by **time, frequency, wavelength**, etc. (TDM, FDM, WDM).

## 2.6 Quick exam checklist

- Always ask: **Is the question asking for $T_t$, $T_p$ or both?**
- RTT $= 2 \times T_p$ ($+ T_t$ of the data and ack if asked).
- Convert **distance** $\rightarrow$ **time** using speed; **size** $\rightarrow$ **time** using bandwidth.
- "Satellite link" $\rightarrow T_p$ is huge ($\approx$ 120–270 ms one-way), $T_t$ small, so utilisation is tiny (see Ch 5).

CHAPTER

# 3

# Data Link Layer (DLL) — Role and Framing

IN THIS CHAPTER


- 3.1 What does the Data Link Layer do?
- 3.2 Framing


## 3.1 What does the Data Link Layer do?

- **Provides service to the network layer** and **encapsulates the IP packet** inside a *frame*.
- TCP/IP says IP must be used at the network layer but **does not fix the data-link protocol**; in practice **Ethernet** is used on LANs.
- DLL provides **hop-to-hop** (**node-to-node**) service.

### Functions

1. **Framing**
2. **Flow control**
3. **Error control**
4. **Media Access Control** (**MAC**) — who may use a shared link now

### Sub-layers

| Link type | Sub-layers needed |
|---|---|
| **Broadcast link** (many senders/receivers on the same medium, e.g. Ethernet bus, Wi-Fi) | **Data Link Control (DLC/LLC) + MAC** |
| **Point-to-point link** (two nodes only) | only **Data Link Control** |

KEY IDEA

The MAC sub-layer decides *who may transmit when*, so that senders avoid collisions on a shared channel (Ch 6).

## 3.2 Framing

On a link a frame is just a **stream of bits**: `0110011010101101000 1 ...` — the receiver must know **where a frame starts and ends**. Because frames can have variable length, we need rules.

**Framing = a way to mark the beginning and the end of a frame.**

| Frame type | How the end is found |
|---|---|
| **Fixed-length** | every frame has the same size (wastes space if frames are small) |
| **Variable-length** | (a) **Frame-length field** at the very beginning (Ethernet-style), or (b) **Flags** (special start/end markers) |

### The problem with flags

Layout: `| Flag | Data | Flag |`. If the **data itself contains the flag pattern**, the receiver is fooled.

FROM YOUR NOTES

*Example:* Flag = `101`, data = `001101001 0101`. Wherever `101` appears inside the data, the receiver thinks a frame starts/ends. Two cures: **byte stuffing** and **bit stuffing**.

## (1) Byte stuffing (character stuffing)

- Whenever the **flag byte** appears in the data → put an **escape character (ESC)** before it.
- If the data itself contains the **ESC** → escape it too (ESC ESC).
- Receiver **de-stuffs**: whenever it sees ESC, it drops it and keeps the next byte as data.

FROM YOUR NOTES

**Example (notes).** Flag = `$`, Escape = `\`, Data = `a0z1_$e\$95` - Data characters: `a 0 z 1 _ $ e \ $ 9 5` - `$` → `\$`, `\` → `\\`, `$` → `\$` - **Escaped data:** `a0z1_\$e\\\$95` - **Frame:** `$ a0z1_\$e\\\$95 $`

## (2) Bit stuffing

- Flag is a fixed bit pattern: **`01111110`** (six 1s between 0s).
- Rule (sender): **after any five consecutive 1s in the data, insert a 0.**
- Receiver: after five consecutive 1s, if the next bit is 0 → **remove it**. (If it is 1 → it is the flag/abort.)

NOTE ON YOUR HANDWRITTEN NOTES

Your notes state it as "stuff a 0 whenever 0 is followed by five 1s". The standard HDLC rule is "**after five consecutive 1s**" — it gives the same result as long as the run is preceded by a 0 and is the safest version to use in the exam.

**Worked example:** Data: `0 1 1 1 1 1 1 0 1 1 1 1 1 0` - First run: `11111` (five 1s) → insert 0 → `11111 0`, then the 6th `1`, then `0` - Second run: `11111` → insert 0 → then `0`

Stuffed: `0 11111 0 1 0 11111 0 0` → **`0111110101111100`** (16 bits; 2 bits added).

EXAM SHORTCUT

**Counting shortcut:** extra bits = number of times a run of five 1s is completed (a run of 10 ones gets 2 stuffed zeros, etc.).

### Comparison

| | Byte stuffing | Bit stuffing |
|---|---|---|
| Unit | character/byte | single bit |
| Overhead | an ESC byte per flag/ESC | 1 bit per five 1s |

| | Byte stuffing | Bit stuffing |
|---|---|---|
| Used in | PPP (byte-oriented) | HDLC |

CHAPTER

# 4

# Error Control

Exam priority: ☐ Tier 4 — low yield

*(but easy 1–2 marks: CRC / Hamming appear in 2021, 2026)*

IN THIS CHAPTER


- 4.1 Types of errors
- 4.2 Dataword → Codeword (block coding)
- 4.3 Hamming distance
- 4.4 Linear block codes
- 4.5 Parity check code
- 4.6 Cyclic codes and CRC
- 4.7 Hamming code (single-error correction) (2021)
- 4.8 Internet checksum (used in IP, TCP, UDP) (see IP header)


## 4.1 Types of errors

1. **Single-bit error** — only 1 bit flips.
2. **Burst error** — a group of bits flip (the burst length is measured from the first to the last corrupted bit).

**Two steps of error control:** (1) **Error detection**, (2) **Error correction** (forward correction or retransmission).

## 4.2 Dataword → Codeword (block coding)

- Sender converts a **k-bit dataword** into an **n-bit codeword** by appending **r = n − k redundant bits**.
- For every dataword there is exactly **one valid codeword**. The receiver knows which codewords are valid.

```
[ k-bit dataword ] → [ k data bits | r check bits ]  (n bits)
```

FROM YOUR NOTES

**Example (notes).** k = 4, r = 2 → n = 6 - valid codewords = $2^k = 16$ - total possible n-bit words = $2^n = 64$ - invalid = 64 − 16 = **48**

TRAP

**An error is detected only if the received word is an invalid codeword.** If a valid codeword turns into *another valid* codeword, the error goes **undetected**.

## 4.3 Hamming distance

**HD(x, y) = number of positions in which two equal-length bit strings differ = number of 1s in (x ⊕ y).**

FROM YOUR NOTES

*Example:* HD(10110, 01101): XOR = 11011 → four 1s → **HD = 4**.

### Minimum Hamming distance ($d_{\min}$) and power of a code

| Goal | Required $d_{\min}$ |
|---|---|
| **Detect** up to $s$ bit errors | $d_{\min} \geq \mathbf{s + 1}$ |
| **Correct** up to $t$ bit errors | $d_{\min} \geq \mathbf{2t + 1}$ |

FROM YOUR NOTES

*Why n+1 for detection (notes):* if two valid codewords are ≤ n apart, then n flips could turn one valid word into another valid word → undetectable. You need ≥ n+1 flips to reach another valid word.

## 4.4 Linear block codes

KEY IDEA

A code is a **linear block code** if the **XOR of any two valid codewords is also a valid codeword.**

EXAM SHORTCUT

**Shortcut:** for a linear block code, $d_{\min}$ **= minimum number of 1s in any non-zero codeword** (the all-zero word is ignored).

FROM YOUR NOTES

**Example (notes).**

| Dataword | Codeword | #1s |
|---|---|---|
| 00 | 000 | 0 |
| 01 | 011 | 2 |
| 10 | 101 | 2 |
| 11 | 110 | 2 |

Ignoring all-zeros: $d_{\min} = 2 \rightarrow$ can **detect up to 1-bit errors** ($s + 1 = 2 \rightarrow s = 1$).

## 4.5 Parity check code

Append **one bit** = XOR of all data bits (even parity → total number of 1s becomes even). - It is a linear block code with $d_{\min} = 2$. - **Detects all odd-numbered bit errors** (1, 3, 5 ...); **cannot detect any even number of errors** (2, 4 ...); **cannot correct** anything. - (The example in 4.4 *is* an even-parity code.)

## 4.6 Cyclic codes and CRC ★

KEY IDEA

**Cyclic code** = linear block code where a **cyclic rotation** of a valid codeword is also valid. **CRC (Cyclic Redundancy Check)** is the most famous one.

## CRC procedure (sender)

1. Sender and receiver agree on a **divisor (generator) g(x)** of n bits (degree n − 1).
2. Append **n − 1 zeros** to the dataword.
3. Divide using **modulo-2 (XOR) division** (no borrow/carry; addition = subtraction = XOR).
4. The **remainder (n − 1 bits)** replaces the appended zeros → **codeword = dataword + remainder**.

## Receiver

Divide the received codeword by the same divisor: - **Remainder ≠ 0 → corrupted.** - **Remainder = 0 → accepted** (may still be corrupted if the error pattern is a multiple of g(x)).

FROM YOUR NOTES

**Worked example (notes).** Dataword = `1001`, Divisor = `1011` (n = 4 → append 3 zeros → `1001000`)

```
Step  working bits       action
1     1001  XOR 1011  =  0010     (leading 1 -> subtract divisor)
2     0100   (bring down 0)  leading 0 -> XOR 0000
3     1000   (bring down 0)  XOR 1011 = 0011
4     0110   (bring down 0)  leading 0 -> XOR 0000 = 0110
Remainder = 110   (quotient 1010)
```

**Codeword = 1001 110 = `1001110`.** Check at receiver: 1001110 ÷ 1011 → remainder `000` ✔.

## Polynomial form

Bits ↔ polynomial: `00101010` = $0{\cdot}x^7+0{\cdot}x^6+1{\cdot}x^5+0{\cdot}x^4+1{\cdot}x^3+0{\cdot}x^2+1{\cdot}x^1+0{\cdot}x^0 = x^5 + x^3 + x$.

FROM YOUR NOTES

**Notes example in polynomial form.** Dataword d(x) = $x^3 + 1$ (=1001), g(x) = $x^3 + x + 1$ (=1011). - Multiply by $x^3$ (append 3 zeros): $x^6 + x^3$ - Divide by g(x) → remainder $x^2 + x$ - Codeword **c(x)** = $x^6 + x^3 + x^2 + x$ (= 1001110) ✔ matches the binary working above.

NOTE ON YOUR HANDWRITTEN NOTES

*(In your notes the dataword line looks like "$x^4 + x^3$", but the division written below it — $x^6 + x^3$ and the final codeword $x^6 + x^3 + x^2 + x$ — corresponds to dataword $x^3 + 1$. The method is identical.)*

## Analysing CRC: what can it catch? ★

Let **d(x)** = dataword, **g(x)** = generator, **c(x)** = codeword (divisible by g), **e(x)** = error pattern. Received = **c(x) + e(x)**. - c(x)/g(x) leaves remainder 0, so the received word is accepted **exactly when e(x) is divisible by g(x).** - **An error pattern is detected ⇔ g(x) does NOT divide e(x).**

| Error type | Condition on g(x) | Reason |
|---|---|---|
| **All single-bit errors** e(x) = $x^i$ | g(x) must have **at least two terms**, one of them the constant term **1** (e.g. $x^3 + 1$, $x^3 + x + 1$) | if g = $x^k$ or g = 1, g divides some $x^i$ |
| **Two isolated single-bit errors** e(x) = $x^j + x^i = x^i(x^{j-i} + 1)$ | g(x) must **not divide** ($x^t$ **+ 1**) for any t up to frame length (e.g. x+1 and $x^4$+1 fail; $x^7$+$x^6$+1 works for normal frame lengths) | |
| **Any odd number of errors** | g(x) must have **(x + 1) as a factor** | e(x) with odd number of terms is never divisible by (x+1) (XOR of an odd number of 1s is 1) |
| **Burst error of length ≤ r** (r = degree of g) | **always detected** | |
| Burst of length r + 1 | missed with probability $1/2^{r-1}$ | |
| Longer bursts | missed with probability 1/2^r | |

FROM YOUR NOTES

**Examples from your notes**

**(a) Which g(x) guarantees detection of all single-bit errors?** options: (1) $x^3$+1 (2) $x^3$ (3) 1 - (1) ✔ has ≥2 terms and a constant term. - (2) $x^3$ only catches e(x)= $x^2$, x, 1; for e(x) = $x^i$, i ≥ 3, $x^3$ divides e(x) → **fails**. - (3) "1" divides every e(x) → fails.

**(b) Which g(x) can detect two isolated single-bit errors?** (a) x+1 (b) $x^4$+1 (c) $x^7$+$x^6$+1 - (a) ✘ e(x) = $x^5$(x+1) = $x^6$+$x^5$ is divisible by x+1. - (b) ✘ e(x) = $x^2$($x^4$+1) = $x^6$+$x^2$ is divisible by $x^4$+1. - (c) ✔.

EXAM SHORTCUT

**Exam rules of thumb - Number of CRC bits** (check bits) = **degree of g(x)** = divisor length − 1. - g(x) has (x+1) as a factor ⇔ g(x) has an **even number of terms** (e.g. $x^4$+$x^3$+x+1 has 4 terms). - A divisor **of the form $x^k$ + 1** detects all single-bit errors and bursts ≤ k.

GATE-STYLE QUESTION

**CRC question:** message `1101`, generator $x^3$ + x + 1 (`1011`). Find the transmitted word. - Append 000 → 1101000 - 1101 ⊕ 1011 = 0110 → bring down → 1100 ⊕ 1011 = 0111 → bring down → 1110 ⊕ 1011 = 0101 → bring down → 1010 ⊕ 1011 = 0001 - Remainder = **001** → transmitted **1101001**.

## 4.7 Hamming code (single-error correction) ☐ (2021)

**Number of check bits r for m data bits:** smallest r with $2^r \geq m + r + 1$. - m = 4 → r = 3 (Hamming (7,4)); m = 7 → r = 4; m = 11 → r = 4.

**Layout (positions 1...n):** check bits sit at **power-of-2 positions** (1, 2, 4, 8 ...); each check bit $p_i$ (even parity) covers all positions whose binary representation has the i-th bit set. - $p_1$ → positions 1, 3, 5, 7, ... - $p_2$ → positions 2, 3, 6, 7, ... - $p_4$ → positions 4, 5, 6, 7, ...

GATE-STYLE QUESTION

**Example: data 1011** ($d_1$ $d_2$ $d_3$ $d_4$ = 1 0 1 1), layout `p1 p2 d1 p4 d2 d3 d4` - $p_1$ = $d_1$⊕$d_2$⊕$d_4$ = 1⊕0⊕1 = 0 - $p_2$ = $d_1$⊕$d_3$⊕$d_4$ = 1⊕1⊕1 = 1 - $p_4$ = $d_2$⊕$d_3$⊕$d_4$ = 0⊕1⊕1 = 0 - Codeword = `0 1 1 0 0 1 1` (**0110011**) - Suppose position 5 flips: received `0110111`. Recompute parity: group(1,3,5,7) = 0,1,1,1 → odd → **1**; group(2,3,6,7) = 1,1,1,1 → even → **0**; group(4,5,6,7) = 0,1,1,1 → odd → **1**. Syndrome ($c_4c_2c_1$) = **101 = 5** → flip position 5. ✔

EXAM SHORTCUT

**Hamming code has $d_{min}$ = 3** → corrects 1 error, or detects 2.

## 4.8 Internet checksum (used in IP, TCP, UDP) (see IP header)

1. Divide data into **16-bit words**; **add them** (**one's-complement addition: carry out is added back**).
2. **Complement** the sum → checksum.
3. Receiver adds all words **including checksum**; the result must be **all 1s** (0xFFFF) → complement = 0.

GATE-STYLE QUESTION

**Example (IP header words, checksum field = 0):** 4500, 0073, 0000, 4000, 4011, 0000, C0A8, 0001, C0A8, 00C7 - Plain sum = 0x2479C (it overflowed 16 bits) → **fold the carry back**: 0x479C + 0x2 = **0x479E** - Checksum = ~0x479E = 0xFFFF − 0x479E = **0xB861** - Receiver check: 0x479E + 0xB861 = 0xFFFF ✔ (all 1s → no error detected)

CHAPTER

# 5

# Flow Control and Sliding Window

Exam priority: ★ Tier 3

IN THIS CHAPTER


- 5.1 Why flow control?
- 5.2 Stop-and-Wait (SW)
- 5.3 Why Sliding Window?
- 5.4 Go-Back-N (GBN)
- 5.5 Selective Repeat (SR)
- 5.6 Efficiency and throughput of sliding window


## 5.1 Why flow control?

If the **sender sends faster than the receiver can receive**, flow control lets the sender **adjust its speed**.

## 5.2 Stop-and-Wait (SW)

**Rule:** send one frame, start a timer, **wait for its ACK**. - ACK arrives before time-out → send next frame. - Timer expires → **resend** the frame and restart the timer.

**Why no ACK can happen:** (1) data lost, (2) ACK lost, (3) data corrupted (receiver discards it → no ACK).

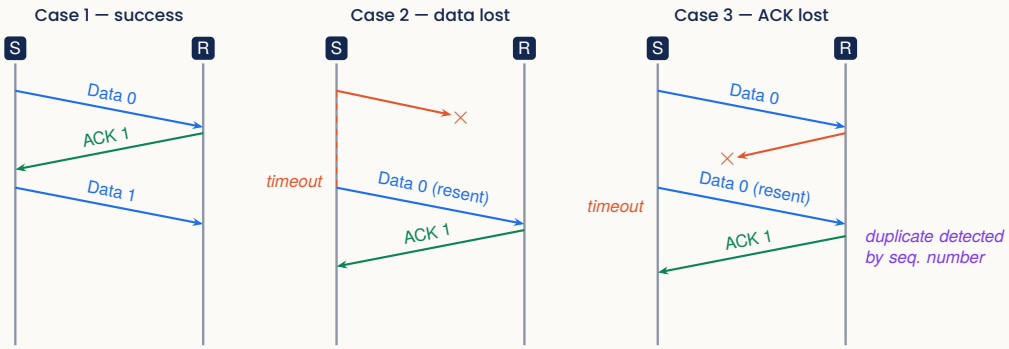



**Figure 5.1** Stop-and-Wait: the three situations (success, data lost, ACK lost).

- **Only 2 sequence numbers (0 and 1)** are needed. "Ack 1" means "send the packet with sequence number 1 next".
- Duplicates (case 3) are recognised by the sequence number.

## Efficiency of Stop-and-Wait ★

ONE FRAME, ONE ROUND TRIP

$$T_{\text{total}} = T_t + T_p + T_{ack} + T_p = T_t + 2T_p + T_{ack}$$

$T_{ack}$ = ACK transmission time — usually negligible, ignore unless given.

**Efficiency ($\eta$) = useful time ÷ total time =** $\frac{T_t}{T_t+2T_p}$

Define $a = \frac{T_p}{T_t}$. Then

KEY FORMULA

$$\eta = \frac{1}{1+2a}$$

- **a** = number of frames that can fit in the channel (one-way) — "how long the pipe is, measured in frames".
- In stop-and-wait we send **1 frame** while the pipe could carry **(2a + 1)** frames in one round trip → $\eta = 1/(1+2a)$.

FROM YOUR NOTES

**Mini example (notes).** $T_p = 10$ min, $T_t = 2$ min → a = 5, the channel holds 5 frames; $\eta = 1/(1+10) = 1/11$.

### Throughput

KEY FORMULA

$$\text{Throughput} = \frac{L}{T_t + 2T_p} = B \times \eta$$

FROM YOUR NOTES

**Example (Earth–Moon link, notes).** 40 Kbps link, Moon is 1.5 light-seconds away, 1 KB packets, stop-and-wait. Data rate and utilisation? - L = 1 KB = 8 Kb; $T_t$ = 8 Kb / 40 Kbps = **0.2 s**; $T_p$ = **1.5 s** - Total = $T_t + 2T_p$ = 0.2 + 3 = **3.2 s** - **Throughput** = 8 Kb / 3.2 s = **2.5 Kbps** - **Efficiency (utilisation)** = 0.2 / 3.2 = 1/16 = **6.25 %** - Check: BW $\times \eta$ = 40 Kbps $\times$ 0.0625 = 2.5 Kbps ✔

### Expected number of transmissions when frames can be lost ★

Let **p** = probability a frame (transmission) fails. Each attempt is independent.

KEY FORMULA

$$E[\text{transmissions per packet}] = \frac{1}{1-p}$$

FROM YOUR NOTES

**Example (notes).** Wireless link, packet error probability 0.2, stop-and-wait. Average transmissions to send **100 packets**? Let E = expected transmissions for one packet. E = 0.8×1 + 0.2×(1 + E) ⇒ 0.8E = 1 ⇒ **E = 1.25** For 100 packets: 100 × 1.25 = **125**.

EXAM SHORTCUT

Effective efficiency with errors ≈ $\eta \times (\mathbf{1} - \mathbf{p})$.

## 5.3 Why Sliding Window?

In stop-and-wait, during the $2T_p$ waiting time **nothing is sent** although many packets could have been. **Sliding window protocols** let the sender send **several packets before the first ACK arrives**: 1. **Go-Back-N (GBN)** 2. **Selective Repeat (SR)** Idea: send multiple packets within the $2T_p$ period.

## 5.4 Go-Back-N (GBN)

- **Sender window = N** packets (can be sent without waiting). **Receiver window = 1.**
- The receiver accepts packets **only in order**; out-of-order packets are **silently discarded**.
- **Cumulative acknowledgement:** an ACK for packet $n$ means **all packets up to n−1 have arrived** (in the notes: "Ack n ⇒ next expected is n").
- A **timer** is started for the (oldest) sent packet.

FROM YOUR NOTES

**Example N = 4, packets 0–9**

**Case 1 – no loss.** Packets 0–3 go out back-to-back; as ACKs return the window slides and 4, 5, ... are sent. Nothing is lost.

**Case 2 – packet 1 lost.** Receiver gets 0, then 2 (not expected) → **discards 2, 3, 4 silently** and keeps re-asking for 1 (Ack 1). When the **timer for packet 1 expires, the sender resends the whole window: 1, 2, 3, 4.**

**Case 3 – ACK lost** (say Ack 2). The sender's timer expires and the sender **resends all packets in the window**. (If a later cumulative ACK arrives in time it covers the lost one and nothing is resent.)

**Case 4 – ACK delayed.** Ack 2 arrives before Ack 1 — this is fine: ACKs are cumulative; receiver sends Ack 2 only after receiving 0 and 1, so the sender can slide past both.

**Corrupted packet:** receiver silently discards it.

### Sequence number size and window size (GBN) ★

Suppose sequence numbers 0–3 (m = 2 bits) and window 4. If all 4 packets are delivered but all 4 ACKs are lost, the sender retransmits 0,1,2,3, but the receiver now **expects a new packet 0** and wrongly **accepts the old copy** → error.

KEY FORMULA

GBN: $N \leq 2^m - 1$

(m = bits in sequence-number field).

(Notes: "window size $< 2^m$".)

## 5.5 Selective Repeat (SR)

- **Sender window = Receiver window = $2^m$ / 2 = $2^{m-1}$.**
- **Individual acknowledgement:** "Ack n" = packet n has been received.
- A **separate timer for every packet**.
- Receiver **accepts out-of-order** packets and stores them in a buffer.

FROM YOUR NOTES

**Example N = 3, packets 0,1,2,3,4,5...**

| Case | What happens |
|---|---|
| **C1 – success** | window slides as ACKs arrive |
| **C2 – packet 1 lost** | packet 2 is still accepted and buffered. When packet 1's timer expires **only packet 1 is resent**, not the whole window |
| **C3 – ACK 1 lost** | Ack 0 → window slides. Ack 2 received → 2 is marked "acked" but the window does **not** slide (1 not acked). Timer for 1 expires → **only packet 1 resent**. On Ack 1 the window slides immediately to **[3,4,5]** |
| **C4 – data delivered out of order** | packet 2 arrives before 1 → receiver accepts & buffers 2 and sends Ack 2; when 1 arrives sends Ack 1 and the sender slides |
| **C5 – ACK delayed** | Ack 2 arrives before Ack 1: P2 marked acknowledged but window doesn't slide until Ack 1 arrives, then it slides to [3,4,5] |
| **C6 – corrupt packet** | silent discard works; optionally a **NAK** can be sent so the sender needn't wait for the timer |

### Why SR window $\leq 2^{m-1}$ ?

Take m = 2 → 4 sequence numbers, and suppose the window is **3** ($> 2^m/2$). Packets 0,1,2 are all received, but their ACKs are lost. The receiver now expects **[3, 0, 1]**. The sender resends 0,1,2 → the receiver wrongly accepts the **old 0 and 1 as new** packets and discards 2.

KEY FORMULA

SR: $N \leq 2^{m-1}$

### General rule for any sliding-window protocol

KEY FORMULA

$$N_S + N_R \leq 2^m$$

(sender window + receiver window $\leq$ number of sequence numbers)

- Stop-and-wait: $1 + 1 = 2 \rightarrow 1$ bit.
- GBN: $N + 1 \leq 2^m \rightarrow N \leq 2^m - 1$.
- SR: $2N \leq 2^m \rightarrow N \leq 2^{m-1}$.

## 5.6 Efficiency and throughput of sliding window

KEY FORMULA

$$\eta = \frac{N_S T_t}{T_t + 2T_p} = \frac{N_S}{1+2a}$$

(cap at 1 = 100 %)

$$\text{Throughput} = \frac{N_S L}{T_t + 2T_p} = N_S \cdot B \cdot \eta_{SW}$$

(where $\eta_{SW} = 1/(1+2a)$)

EXAM SHORTCUT

**Window needed to keep the pipe full:** $N_S = 1 + 2a$.

EXAM SHORTCUT

**Sequence-number bits needed to achieve 100 % efficiency:** - GBN: need $1+2a$ window $\rightarrow m = \lceil \log_2(1+2a+1) \rceil = \lceil \log_2(2+2a) \rceil$ - SR: need $(1+2a)$ in *each* window $\rightarrow m = \lceil \log_2(2(1+2a)) \rceil$

### Comparison table

| | Stop & Wait | Go-Back-N | Selective Repeat |
|---|---|---|---|
| Sender window | 1 | N ($\leq 2^m - 1$) | N ($\leq 2^{m-1}$) |
| Receiver window | 1 | 1 | N |
| Out-of-order packets | – | **discarded** | **buffered** |
| ACK type | – | **cumulative** | **individual** |
| On loss, resend | 1 packet | **entire window** | **only lost packet** |
| Timers | 1 | 1 (oldest) | 1 per packet |
| Receiver complexity | lowest | low | high (buffer + sorting) |

EXAM SHORTCUT

**Expected transmissions with loss probability p (bonus):** SR & SW: $1/(1-p)$. GBN: $(1 - p + N{\cdot}p)/(1 - p)$.

GATE-STYLE QUESTION

**Example.** 1 Mbps link, one-way $T_p = 10$ ms, frame = 1000 bits ($T_t = 1$ ms) - $a = 10$; **SW $\eta$ = 1/21 $\approx$ 4.76 %**, throughput $\approx$ 47.6 Kbps - To fill the pipe: **N = 21 frames** - GBN needs $N + 1 = 22 \leq 2^m \rightarrow$ **m = 5 bits**; SR needs $2{\times}21 = 42 \leq 2^m \rightarrow$ **m = 6 bits** - GBN with $N = 7$: $\eta = 7/21 =$ **33.3 %**

CHAPTER

# 6

# MAC Layer, Ethernet and Devices

Exam priority: ★ Tier 3

IN THIS CHAPTER


- 6.1 The multiple-access problem
- 6.2 ALOHA (formulas are favourites)
- 6.3 CSMA — Carrier Sense Multiple Access
- 6.4 CSMA/CD — Collision Detection (Ethernet)
- 6.5 Binary Exponential Backoff algorithm
- 6.6 CSMA/CA (Wi-Fi) — bonus
- 6.7 Ethernet frame (IEEE 802.3)
- 6.8 Controlled access and channelization (theory, 1-markers)
- 6.9 Hubs, bridges, switches, routers — domains


## 6.1 The multiple-access problem

When **many stations share one channel**, two sending at once **collide**. **Multiple Access Protocols** decide who talks when:

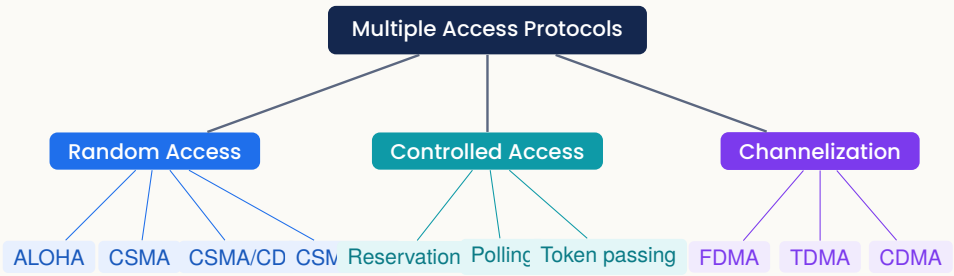



**Figure 6.1** Taxonomy of multiple-access protocols.

**Random access** ( notes): *all hosts can send; no host is superior to another.*

FROM YOUR NOTES

**Four questions every MAC protocol must answer** 1. **When** can a station access the medium? 2. **What** can a station do if the medium is busy? 3. **How** can a station determine success/failure of its transmission? 4. **What** can a station do in case of a collision?

## 6.2 ALOHA ★ (formulas are favourites)

Let **Tfr** = frame transmission time and **G** = average number of frames *generated per frame-time* (offered load). **S** = throughput (successful frames per frame-time).

| | Pure ALOHA | Slotted ALOHA |
|---|---|---|
| Rule | send whenever you have data; if no ACK, wait random time and resend | time divided into slots of length Tfr; send only **at slot start** |
| **Vulnerable time** | **2 × Tfr** | **Tfr** |
| **Throughput** | $\mathbf{S = G \cdot e^{-2G}}$ | $\mathbf{S = G \cdot e^{-G}}$ |
| Max throughput | at **G = 0.5**: $S = 1/(2e) \approx$ **0.184 (18.4 %)** | at **G = 1**: $S = 1/e \approx$ **0.368 (36.8 %)** |

GATE-STYLE QUESTION

**Example.** 1 Mbps channel, 1000-bit frames (Tfr = 1 ms). All stations together create **500 frames/s** under pure ALOHA. - $G = 500 \times 0.001 = 0.5 \rightarrow$ $S = 0.5 \cdot e^{-1} = 0.1839$ frames per frame-time - Successful frames/s = 0.1839 $\times$ 1000 = **183.9 frames/s** $\approx$ 184 Kbps.

EXAM SHORTCUT

*Slotted ALOHA doubles the best-case efficiency of pure ALOHA.*

## 6.3 CSMA — Carrier Sense Multiple Access

**Listen before you talk.** Senses the carrier; if idle → transmit. Collisions can still happen because of **propagation delay** (A's signal hasn't reached B yet).

| Persistence method | When channel is busy |
|---|---|
| **1-persistent** | keep sensing; send immediately (probability 1) when idle |
| **Non-persistent** | wait a random time, then sense again |
| **p-persistent** (slotted channel) | when idle, send with probability p; else wait for the next slot |

## 6.4 CSMA/CD — Collision Detection (Ethernet) ★

- Each host **monitors** the medium. **Only if the channel is free** does it start transmitting.
- **After starting, it keeps sensing while it is transmitting** to see if its transmission was successful (a collision changes the voltage). A host can sense the channel **only as long as it is transmitting**.
- On collision a **jamming signal** is sent so **all hosts know** about the collision.
- **There is no acknowledgement system in CSMA/CD** (success = no collision detected during transmission).

### Why minimum frame size? ★

Suppose A sends; the collision happens at the far end; the jam signal must **travel back to A**. If A had already finished sending the whole frame and stopped sensing, it would never know about the collision.

Worst case: B starts just before A's signal arrives (after $\approx T_p$) → collision signal needs another $T_p$ to return to A → $\mathbf{2}T_p$.

KEY FORMULA

$$T_t \geq 2\,T_p \quad \Rightarrow \quad L_{\min} = 2\,T_p\,B$$

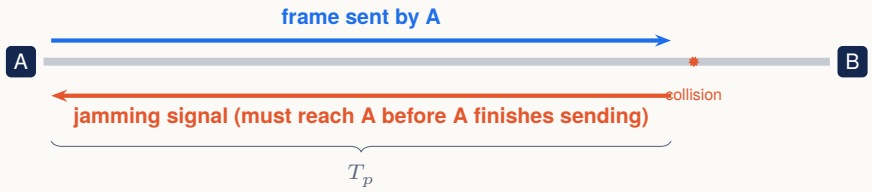



**Figure 6.2** Why a frame must last at least 2Tp: the jam signal has to reach the sender while it is still transmitting.

GATE-STYLE QUESTION

**Examples** 1. BW = 10 Mbps, cable 2 km, v = $2\times10^8$ m/s $\rightarrow$ $T_p = 10$ µs $\rightarrow$ $L_{\text{min}}$ **= 2 × 10 µs** × $10^7$ **= 200 bits**. 2. 1 Gbps, min frame 512 bits $\rightarrow$ $T_t = 512$ ns $\rightarrow 2T_p \leq 512$ ns $\rightarrow T_p \leq 256$ ns $\rightarrow$ **max length = 256 ns** × **2**×$10^8$ **= 51.2 m** (that is why Gigabit Ethernet uses "carrier extension" to enlarge frames).

EXAM SHORTCUT

*If $L_{\text{min}}$ is fixed, higher bandwidth ⇒ shorter maximum cable; longer cable ⇒ larger minimum frame.*

**Efficiency of CSMA/CD (bonus formula):** $\eta \approx 1 \,/\, (1 + 6.44\text{a})$, a = $T_p/T_t$.

## 6.5 Binary Exponential Backoff algorithm ★

After a collision, how long should a host wait before retransmitting? The algorithm **picks a random wait time** so that the colliding hosts are unlikely to collide again.

**Standard rule (use this in the exam):** after the **K-th collision** of the *same frame*, choose an integer **r uniformly from {0, 1, 2, ..., $2^K - 1$}** and wait **r** × **slot time** (slot time = $2T_p$; in 10 Mbps Ethernet = 51.2 µs). The exponent is capped at 10, and after **16** attempts the frame is dropped.

FROM YOUR NOTES

**Notes example (A and B collide).** K = 1 for both $\rightarrow$ each chooses from **{0, 1}**.

| A | B | Outcome |
|---|---|---|
| 0 | 0 | collision |
| 0 | 1 | **A wins** |
| 1 | 0 | **B wins** |
| 1 | 1 | collision |

P(collision again) $= 2/4 =$ **1/2**.

FROM YOUR NOTES

**Next scenario in the notes:** A wins, sends its packet and now wants to send the **next packet** (fresh frame $\rightarrow$ K = 1 $\rightarrow$ {0,1}); B is still on its **second** collision. - **Standard version** (B: K = 2 $\rightarrow$ {0,1,2,3}); 8 equally likely pairs: - A wins (A's number < B's): (0,1),(0,2),(0,3),(1,2),(1,3) = **5/8** - B wins: (1,0) = **1/8** - collision: (0,0),(1,1) = **2/8 = 1/4** - Notes version (B's range written as {0,1,2}, 6 pairs): P(collision) = 2/6 = **1/3**, P(A wins) = 3/6 = **1/2**, P(B wins) = 1/6.

TRAP

**Use the standard {0 ... $2^K-1$} range in GATE.** (The classic GATE question on this has answer **5/8** for "A wins the second race".) The notes' narrower range gives 1/2 — the *method* (count favourable pairs / total pairs) is identical.

Key observation (notes): **the probability of collision decreases over time** as the window doubles. Also notice that the host that just *won* (A) restarts with a small window while the loser (B) has a bigger window $\rightarrow$ B tends to lose again ("capture effect").

## 6.6 CSMA/CA (Wi-Fi) — bonus □

Wireless nodes **cannot detect collisions while sending**, so they **avoid** them: wait **IFS**, then a random backoff; optional **RTS/CTS** handshake; **ACK is mandatory** (unlike CSMA/CD).

## 6.7 Ethernet frame (IEEE 802.3) ★

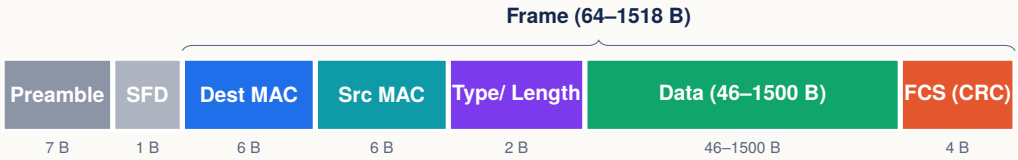



**Figure 6.3** The Ethernet (IEEE 802.3) frame.

- **MAC address = 48 bits** (6 bytes, written in hex e.g. `3A:1F:00:7C:92:B5`).
- **Payload 46 to 1500 bytes.** 1500 B = **MTU** of Ethernet. Frame (DA to FCS) = **64 to 1518 B**. **Minimum frame = 64 B = 512 bits** → slot time at 10 Mbps = **51.2 µs**.
- FCS = **CRC-32** (32 check bits).
- Padding is added if payload < 46 B.

## 6.8 Controlled access and channelization (theory, 1-markers) □

| Method | Idea |
|---|---|
| **Reservation** | stations reserve a time slot in a reservation interval before data intervals |
| **Polling** | a primary station asks each secondary "do you have data?" |
| **Token passing** | a special **token** circulates; only the token holder may send |
| **FDMA** | channel split by **frequency** bands |
| **TDMA** | split by **time** slots |
| **CDMA** | all use the same band **simultaneously**, separated by orthogonal **codes** |

## 6.9 Hubs, bridges, switches, routers — domains ★

| Device | Layer | Collision domains | Broadcast domains |
|---|---|---|---|
| **Hub / repeater** | 1 | whole hub = **1** collision domain | 1 |

| Device | Layer | Collision domains | Broadcast domains |
|---|---|---|---|
| **Bridge / switch** | 2 | **each port = 1 collision domain** | **1 for the whole switch** (without VLANs) |
| **Router** | 3 | each port separate | **each port separate broadcast domain** |

**Learning switch (transparent bridge):** keeps a table *MAC* $\rightarrow$ *port*. For each incoming frame: **learn** the source MAC and its port; if destination known $\rightarrow$ **forward only to that port**; if destination in the same port $\rightarrow$ **filter (drop)**; if unknown or broadcast $\rightarrow$ **flood** to all other ports.

GATE-STYLE QUESTION

*A switch connects Hub-1 (4 hosts), Hub-2 (3 hosts) and one directly attached host.* Collision domains = **3** (Hub-1 side, Hub-2 side, direct link); broadcast domains = **1**.

CHAPTER

# 7

# IP Addressing, Classes, Subnetting, VLSM

Exam priority: ★★★ Tier 1 — sure-shot

IN THIS CHAPTER


- 7.1 IPv4 address basics
- 7.2 Classful addressing
- 7.3 Classless addressing (CIDR notation)
- 7.4 Subnetting
- 7.5 VLSM — Variable-Length Subnet Masks


KEY IDEA

This chapter + Ch 8 together are the **#1 scoring area** of CN ($\approx 1.7$ marks/paper). Learn the **"block-size trick"** below and you will solve every question in under a minute.

## 7.1 IPv4 address basics

- An IPv4 address is **4 bytes = 32 bits**, written in **dotted decimal**: `192.168.9.0` (each byte 0–255).
- Each IP address has **two parts: Network ID (prefix)** and **Host ID (suffix)**.

```
|<------ network ID ------>|<---- host ID ---->|
```

- **Network ID** identifies the network; **Host ID** identifies the machine inside it.
- If **n** bits are the network ID $\rightarrow 2^n$ **possible networks**, and **32 − n** host bits $\rightarrow 2^{32-n} - 2$ **usable hosts** ($-2$ because the first address = network address and the last = broadcast address).

FROM YOUR NOTES

*Example (notes):* 8 bits for network ID $\rightarrow 2^8 = 256$ networks, hosts $= 2^{24} - 2$ each. **This rigid scheme allows only 256 networks** — which led to flexible schemes.

### Binary ↔ decimal helper (memorise)

| bit weight | 128 | 64 | 32 | 16 | 8 | 4 | 2 | 1 |
|---|---|---|---|---|---|---|---|---|
| **leading 1s** $k$ | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 |
| **mask octet** | **128** | **192** | **224** | **240** | **248** | **252** | **254** | **255** |
| **block size** $= 256 -$ **mask** | **128** | **64** | **32** | **16** | **8** | **4** | **2** | **1** |

## 7.2 Classful addressing

The first bits decide the class (so the **prefix length is implicit**).

| Class | Leftmost bits | Network bits | Range (1st octet) | Full range | Default mask | #Network | #Hosts/netwo |
|---|---|---|---|---|---|---|---|
| **A** | **0** | 8 | 0 – 127 | 0.0.0.0 – 127.255.255.255 | 255.0.0.0 (/8) | $2^7 = 128$ | $2^{24} - 2 =$ 16,777,214 |
| **B** | **10** | 16 | 128 – 191 | 128.0.0.0 – 191.255.255.255 | 255.255.0.0 (/16) | $2^{14} =$ 16,384 | $2^{16} - 2 =$ 65,534 |

| Class | Leftmost bits | Network bits | Range (1st octet) | Full range | Default mask | #Network | #Hosts/netwo |
|---|---|---|---|---|---|---|---|
| C | **110** | 24 | 192 – 223 | 192.0.0.0 – 223.255.255.255 | 255.255.255.0 (/24) | $2^{21} = 2{,}097{,}152$ | $2^8 - 2 = 254$ |
| D | **1110** | – | 224 – 239 | **Multicast** (no network/host split) | – | – | – |
| E | **1111** | – | 240 – 255 | **Reserved / experimental** | – | – | – |

EXAM SHORTCUT

**Class by first octet:** $<128 \rightarrow$ A, 128–191 $\rightarrow$ B, 192–223 $\rightarrow$ C, 224–239 $\rightarrow$ D, $\geq 240 \rightarrow$ E. (0 and 127 are special in class A, so usable A networks = 126.)

## Special / reserved addresses ★

| Address | Meaning |
|---|---|
| **Network address** (host bits all 0), e.g. 192.168.9.0/24 | **Identifies the network**; cannot be given to a host. |
| **Direct Broadcast Address (DBA)** (host bits all 1), e.g. 192.168.9.255/24 | The **last address** — broadcasts **within that network**; cannot be given to a host. A packet from **another network** can be sent to a DBA to be broadcast inside that network. |
| **Limited broadcast** `255.255.255.255` | Broadcast **within the local network only**; routers **never forward** it. |
| `0.0.0.0` | "This host" — used by a host that does not yet know its IP (e.g. while booting/DHCP). Not usable for routing/destination. Also default route in tables. |
| `127.0.0.0/8` (e.g. 127.0.0.1) | **Loopback** — packets never leave the machine |
| Private ranges (not routed on Internet) | `10.0.0.0/8`, `172.16.0.0/12` (172.16–172.31), `192.168.0.0/16` |
| `169.254.0.0/16` | Link-local / APIPA (when DHCP fails) |
| 224.0.0.0 – 239.255.255.255 | Multicast (class D) |

## 7.3 Classless addressing (CIDR notation)

Classful was wasteful. In **classless** addressing the **prefix length is not implicit** — you write it as **IP/n** (slash notation).

KEY IDEA

**a.b.c.d / n → n bits** are the network (prefix), **32 − n** bits are the host.

FROM YOUR NOTES

**Example (notes).** `12.24.34.127 / 21` - 21 bits for prefix, 32 − 21 = **11 bits for host** - 34 = `00100010` → the 21-bit prefix takes the first 5 bits of the 3rd octet: `00100|010` - **First address (network):** `12.24.00100000.0` = **12.24.32.0** - **Last address (broadcast):** `12.24.00100111.255` = **12.24.39.255**

### The Block-Size Trick (fast method, no binary)

For **IP / n**: 1. Find the **"interesting octet"** — the octet where the mask is neither 0 nor 255. (n=21 → mask 255.255.248.0 → 3rd octet.) 2. **Block size = 256 − (mask value in that octet)** (here 256 − 248 = **8**). 3. **Network address** = largest multiple of block size ≤ IP's value in that octet (34 → 32). Octets after it → 0. 4. **Broadcast** = Network + (block size − 1) in that octet, octets after → 255 (32 + 7 = 39 → **12.24.39.255**). 5. **Usable hosts** = $2^{32-n} - 2$.

FROM YOUR NOTES

**Notes example 2:** `220.8.24.255 / 25` → mask 255.255.255.128 → block = 128 (4th octet). 255 → multiple of 128 ≤ 255 = 128. **First = 220.8.24.128, Last = 220.8.24.255.** (Hosts: $2^7 - 2 = 126$.)

FROM YOUR NOTES

**Notes example 3:** `167.199.170.82 / 27` → mask ...224 → block 32. 82 → 64. **First = 167.199.170.64, Last = 167.199.170.95. Hosts = $2^5 - 2$ = 30.** Binary check: 82 = 01010010 → first 3 bits `010` then host bits → 010|00000 = 64 to 010|11111 = 95 ✔.

### Subnet mask

Mask = n ones then (32 − n) zeros. Network ID = **IP AND Mask**. Broadcast = **Network OR (NOT Mask)**.

| /n | Mask | Block | Total addresses | Usable hosts |
|---|---|---|---|---|
| /8 | 255.0.0.0 | – | 16,777,216 | 16,777,214 |
| /16 | 255.255.0.0 | – | 65,536 | 65,534 |
| /20 | 255.255.240.0 | 16 (3rd) | 4,096 | 4,094 |
| /22 | 255.255.252.0 | 4 (3rd) | 1,024 | 1,022 |
| /23 | 255.255.254.0 | 2 (3rd) | 512 | 510 |
| /24 | 255.255.255.0 | – | 256 | 254 |
| /25 | 255.255.255.128 | 128 | 128 | 126 |
| /26 | 255.255.255.192 | 64 | 64 | 62 |
| /27 | 255.255.255.224 | 32 | 32 | 30 |
| /28 | 255.255.255.240 | 16 | 16 | 14 |
| /29 | 255.255.255.248 | 8 | 8 | 6 |
| /30 | 255.255.255.252 | 4 | 4 | 2 |
| /31 | 255.255.255.254 | 2 | 2 | 2 (point-to-point links) |
| /32 | 255.255.255.255 | 1 | 1 | single host route |

### Are two IPs in the same network?

Compute **IP AND mask** for both; equal ⇒ same network (or: they must share the first $n$ bits).

GATE-STYLE QUESTION

*Example:* 10.1.1.5 and 10.1.2.5 — largest prefix for which they are in the same network? Third octets 00000**001** and 00000**010** differ from bit 7 of that octet → common prefix = 16 + 6 = **22 bits** → they are in the same network iff **n** ≤ **22**.

## 7.4 Subnetting

KEY IDEA

**Subnetting = dividing a network into smaller networks.** We **borrow bits from the host part** to make a **subnet ID**.

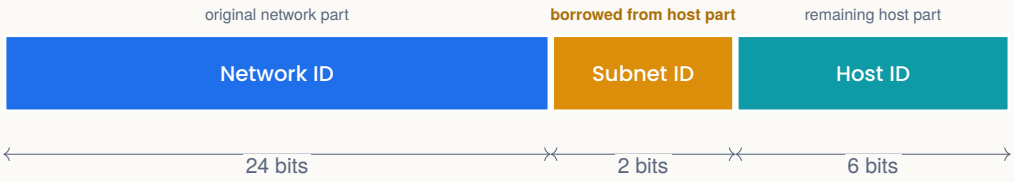



**Figure 7.1** Subnetting borrows bits from the host part to form a subnet ID.

## Fixed-size subnetting

Borrowing **b** bits $\rightarrow 2^b$ **subnets**, each with $2^{hostbits} - 2$ hosts; new prefix = old prefix + b.

FROM YOUR NOTES

**Example 1 (notes).** Divide `192.168.9.0/24` into **four** networks. - Need 2 bits ($2^2 = 4$) $\rightarrow$ new prefix /26, **6 host bits**. - Subnets (block = 64):

| Subnet | Range | Network ID | Broadcast |
|---|---|---|---|
| Sub1 | 192.168.9.0/26 – 192.168.9.63/26 | 192.168.9.0 | 192.168.9.63 |
| Sub2 | 192.168.9.64/26 – 192.168.9.127/26 | 192.168.9.64 | 192.168.9.127 |
| Sub3 | 192.168.9.128/26 – 192.168.9.191/26 | 192.168.9.128 | 192.168.9.191 |
| Sub4 | 192.168.9.192/26 – 192.168.9.255/26 | 192.168.9.192 | 192.168.9.255 |

**Hosts per subnet = $2^6 - 2 = 62$.**

FROM YOUR NOTES

**Example 2 (notes).** Divide `192.168.9.0/24` so that **each network has $\geq$ 10 hosts**. - 8 host bits currently. With 2 bits $\rightarrow 2^2 - 2 = 2$ hosts; 3 bits $\rightarrow$ 6 hosts; **4 bits $\rightarrow$ 14 hosts ✔**. - **4 host bits kept; 4 bits used for subnet ID** $\rightarrow$ new prefix **/28** $\rightarrow 2^4$ **= 16 subnets**: 192.168.9.0/28, .16/28, .32/28 … .240/28.

KEY FORMULA

$$2^h - 2 \;\geq\; \text{hosts required}, \qquad \text{prefix} = 32 - h$$

Choose the smallest host-bit count $h$ that fits.

### The "first subnet / last subnet" confusion

FROM YOUR NOTES

*Notes question:* `192.168.9.0/24` is divided into four subnets and an **external router** forwards a packet to the network. How do we know if the packet is for **subnet 1** or **for the network itself**? - The network ID `192.168.9.0` and Sub-1's network ID `192.168.9.0` are **identical**. **The packet is delivered to the *network*, not to the subnet** — the outside world cannot tell. - Similarly, if the destination is **192.168.9.255** it is the **broadcast address of the whole network**, so it is **broadcast to the whole network, not just the last subnet**.

(That is why old classful rules avoided the first and last subnet; modern CIDR allows them, but be aware of this ambiguity.)

## 7.5 VLSM — Variable-Length Subnet Masks ★★★

**Different subnets get different sizes** according to need. Fixed-size subnetting wastes addresses; VLSM minimises waste.

### Algorithm (always works)

1. **Sort requirements in DECREASING order** of hosts. (Largest first — keeps blocks aligned.)
2. For each requirement find h = smallest number of host bits with $2^h - 2 \geq$ hosts $\rightarrow$ **prefix = 32 − h**.
3. Allocate **consecutively from the start of the address space**, each block **starting at a multiple of its own size**.
4. Remaining unused space can be used later.

FROM YOUR NOTES

**Example 3 (notes).** `14.24.74.0/24` $\rightarrow$ 3 subnets with **120, 60, 10 hosts**. We go in decreasing order: $120 > 60 > 10$. (8 host bits available.)

| Subnet | Hosts needed | Host bits h ($2^h-2$) | Prefix | Range | Usable hosts |
|---|---|---|---|---|---|
| 1 | 120 | 7 (126) | **/25** | 14.24.74.**0**/25 – 14.24.74.**127**/25 | 126 |
| 2 | 60 | 6 (62) | **/26** | 14.24.74.**128**/26 – 14.24.74.**191**/26 | 62 |
| 3 | 10 | 4 (14) | **/28** | 14.24.74.**192**/28 – 14.24.74.**207**/28 | 14 |

Binary view of the starting bits (as in the notes): Subnet 1 → `0xxxxxxx`; Subnet 2 → `10xxxxxx`; Subnet 3 → `1100xxxx`. Free space left: 14.24.74.208 – 14.24.74.255. (*The notes' "pie chart": half the space ($2^7$) for the 0-prefix, a quarter ($2^6$) for the 10-prefix, $2^4$ for the 1100-prefix.*)

FROM YOUR NOTES

**Example 4 (notes).** `172.16.0.0/16` → subnets with **500, 200, 100, 62, 22** hosts. (16 host bits.)

| Subnet | Hosts | Host bits (usable) | Prefix | Range |
|---|---|---|---|---|
| S1 | 500 | 9 (510) | **/23** | 172.16.0.0/23 – 172.16.1.255/23 |
| S2 | 200 | 8 (254) | **/24** | 172.16.2.0/24 – 172.16.2.255/24 |
| S3 | 100 | 7 (126) | **/25** | 172.16.3.0/25 – 172.16.3.127/25 |
| S4 | 62 | 6 (62) | **/26** | 172.16.3.128/26 – 172.16.3.191/26 |
| S5 | 22 | 5 (30) | **/27** | 172.16.3.192/27 – 172.16.3.223/27 |

GATE-STYLE QUESTION

**Extra VLSM practice.** `192.168.1.0/24` with requirements 100, 50, 25, 10, and a point-to-point link (2): - 100 → /25 → 192.168.1.0/25 (0–127) - 50 → /26 → 192.168.1.128/26 (128–191) - 25 → /27 → 192.168.1.192/27 (192–223) - 10 → /28 → 192.168.1.224/28 (224–239) - 2 → /30 → 192.168.1.240/30 (240–243); free: 244–255.

EXAM SHORTCUT

**VLSM check:** a block of $2^h$ addresses must start at an address divisible by $2^h$. (e.g. /26 blocks start at 0, 64, 128, 192.)

TRAP

**Traps** - "Hosts" means *usable*, so add 2 (network + broadcast) when sizing: 62 hosts $\rightarrow$ 64 addresses $\rightarrow$ /26 (62 exactly fits, 63 would need /25). - Do **not** forget routers' interfaces also consume host addresses if the question says so. - Largest-first allocation avoids misalignment.

CHAPTER

# 8

# Forwarding, Longest-Prefix Match, CIDR and Supernetting

Exam priority: ★★★ Tier 1 — sure-shot

IN THIS CHAPTER


- 8.1 The routing (forwarding) table
- 8.2 Forwarding algorithm
- 8.3 Supernetting / route aggregation (CIDR)
- 8.4 CIDR block allocation by an ISP (GATE style)
- 8.5 NAT (bonus, for “IP-header facts” 1-markers)


## 8.1 The routing (forwarding) table

Each router stores at least these **mandatory fields** for every row: 1. **Network ID** 2. **Prefix length** (or mask) 3. **Output interface** (and often *next-hop* IP)

A special row **Default** (0.0.0.0/0) is used when nothing else matches.

## 8.2 Forwarding algorithm ★★★

When a packet with destination IP **A** arrives at a router, the router examines the table **in descending order of prefix length** and for each row: 1. Computes **A′ = A AND mask(prefix length of that row)**. 2. **Compares A′ with the row’s Network ID.** 3. **If equal → forward to that row’s interface** and stop. 4. Else go to the **next row**. If **no match** is found → forward on the **default interface**.

KEY IDEA

This is called the **Longest Prefix Match (LPM)** rule: *among all rows that match, the one with the longest prefix wins.*

FROM YOUR NOTES

**Example (notes).** Router X with four attached networks and a default route:

| S.No | Network ID | Prefix | Interface |
|---|---|---|---|
| 1 | 130.70.65.192 | 26 | m2 |
| 2 | 130.70.65.128 | 25 | m0 |
| 3 | 201.4.22.0 | 24 | m3 |
| 4 | 201.4.16.0 | 22 | m1 |
| 5 | default | – | m4 |

**Destination 201.4.22.128** - Row 1 (/26): 201.4.22.128 & 255.255.255.192 = 201.4.22.128 $\neq$ 130.70.65.192 $\rightarrow$ ✘ - Row 2 (/25): result 201.4.22.128 $\neq$ 130.70.65.128 $\rightarrow$ ✘ - Row 3 (/24): 201.4.22.128 & 255.255.255.0 = **201.4.22.0** ✔ $\rightarrow$ **forward to m3**.

More destinations for the same table (GATE-Q practice):

| Destination | Matching rows | Answer |
|---|---|---|
| 130.70.65.200 | row 1: 200 & 192 = 192 ✔ (also row 2 ✔) $\rightarrow$ longest = /26 | **m2** |
| 130.70.65.130 | row 1: 130&192 = 128 $\neq$ 192 ✘; row 2: 130&128 = 128 ✔ | **m0** |
| 201.4.17.5 | row 3: 201.4.17.0 $\neq$ 201.4.22.0 ✘; row 4 (/22): 17 & 252 = 16 $\rightarrow$ 201.4.16.0 ✔ | **m1** |
| 8.8.8.8 | no match | **m4 (default)** |

GATE-STYLE QUESTION

**Overlapping prefixes (why LPM is needed).**

| Prefix | | Interface | Note |
|---|---|---|---|
| `0.0.0.0/0` | → | **I1** | default route |
| `10.0.0.0/8` | → | **I2** | |
| `10.1.0.0/16` | → | **I3** | |
| `10.1.2.0/24` | → | **I4** | |
| `10.1.2.128/25` | → | **I5** | most specific |

| Dest | Matches | Winner |
|---|---|---|
| 10.1.2.130 | /0, /8, /16, /24, /25 (130 & 128 = 128 ✔) | **I5** |
| 10.1.2.100 | /0, /8, /16, /24 (/25 ✘: 100&128 = 0) | **I4** |
| 10.1.9.9 | /0, /8, /16 | **I3** |
| 10.200.1.1 | /0, /8 | **I2** |
| 11.0.0.1 | /0 | **I1** |

EXAM SHORTCUT

**Exam trick:** Check the **most specific (longest) rows first** and stop at the first match. For one destination you only need **AND with that row's mask** — use the block-size trick instead of binary.

## 8.3 Supernetting / route aggregation (CIDR) ★

Many routers store **several rows that all go to the same interface**. We can **merge them into one row** with a shorter prefix.

FROM YOUR NOTES

**Example (notes).** Router $R_2$ is connected to four /26 networks: 140.24.7.0/26, 140.24.7.64/26, 140.24.7.128/26, 140.24.7.192/26 — all reachable through interface **m0** of router $R_1$, default through m1. $R_1$'s table has 5 rows:

| S.No | Network ID | Prefix | Interface |
|---|---|---|---|
| 1 | 140.24.7.0 | 26 | m0 |
| 2 | 140.24.7.64 | 26 | m0 |
| 3 | 140.24.7.128 | 26 | m0 |
| 4 | 140.24.7.192 | 26 | m0 |
| 5 | Default | – | m1 |

Redundancy! Combine the four m0 rows → **140.24.7.0/24** → **m0**. New table: **row 1: 140.24.7.0/24 → m0; row 2: Default → m1.**

KEY IDEA

**Supernetting doesn't actually merge organisations**; it only **reduces redundancy in a particular router's table** (here router $R_1$).

## Conditions to aggregate k blocks into one

1. The blocks must be **contiguous** (no gaps).
2. k must be a **power of 2**.
3. The **first block's network address must be divisible by the total size** of the merged block (i.e. it must fall on the merged block's boundary). New prefix = old prefix − $\log_2(k)$.

EXAM SHORTCUT

*Examples* - 200.1.0.0/24, 200.1.1.0/24, 200.1.2.0/24, 200.1.3.0/24 → **200.1.0.0/22** ✔ (third octets 0–3, 0 divisible by 4). - 200.1.1.0/24 + 200.1.2.0/24 → ✘ **cannot** be a /23 (starting 1 is not divisible by 2; /23 would be 200.1.0.0–1.255 or 200.1.2.0–3.255). - 192.24.0.0/21, 192.24.8.0/21, 192.24.16.0/21, 192.24.24.0/21 → 4×2048 = 8192 addresses → **192.24.0.0/19** ✔.

## Longest-prefix match with aggregation

FROM YOUR NOTES

**Example (notes).** A second router $R_2$:

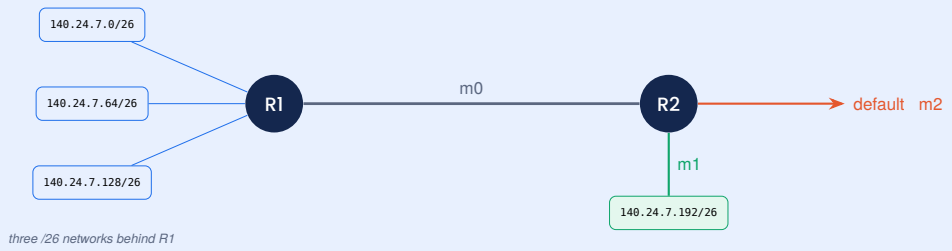



**Figure 8.1** Router R2 attaches one /26 and reaches the other three via R1.

$R_2$'s table:

| S.No | Network ID | Prefix | Interface |
|---|---|---|---|
| 1 | 140.24.7.192 | 26 | m1 |
| 2 | 140.24.7.0 | 24 | m0 |
| 3 | Default | – | m2 |

A packet arrives with destination **140.24.7.193**. Applying each mask it **matches both row 1 and row 2** (193 & 192 = 192 → 140.24.7.192 ✔; 193 & 0 → 140.24.7.0 ✔). **The packet is forwarded to the longest-prefix match → /26 → m1.** ✔

KEY IDEA

This is why the aggregated /24 on m0 does **not** wrongly capture 140.24.7.192–255: the **more specific /26** wins.

## 8.4 CIDR block allocation by an ISP ★ (GATE style)

An ISP owns **190.100.0.0/16** and must serve four customers needing **1000, 500, 250 and 100 hosts**. Decreasing order, aligned:

| Customer | Hosts | Block size | Prefix | Allocation |
|---|---|---|---|---|
| A | 1000 | 1024 | /22 | 190.100.0.0/22 (0.0 – 3.255) |
| B | 500 | 512 | /23 | 190.100.4.0/23 (4.0 – 5.255) |
| C | 250 | 256 | /24 | 190.100.6.0/24 |
| D | 100 | 128 | /25 | 190.100.7.0/25 (7.0 – 7.127) |

ISP can **advertise** a single aggregate **190.100.0.0/16** to the Internet; internally it keeps the four specific routes.

GATE-STYLE QUESTION

**Hole-punching with LPM.** ISP block **205.16.0.0/20** (third octet 0–15). Allocations: X = 205.16.0.0/21 (0–7), Y = 205.16.8.0/22 (8–11), Z = 205.16.12.0/22 (12–15). Destination 205.16.9.200: matches the /20 aggregate and Y's /22 → **goes to Y**.

## 8.5 NAT (bonus, for "IP-header facts" 1-markers) □

**Network Address Translation:** a router/gateway maps **private** addresses (10.x, 172.16–31.x, 192.168.x) to **one public address**, using **(IP, port)** pairs in a translation table. Private addresses are **not routable** on the Internet. NAT changes the **IP header (src IP/checksum) and TCP/UDP port + checksum**.

CHAPTER

# 9

# IP Header, Fragmentation, ICMP and ARP

Exam priority: ★ Tier 3

IN THIS CHAPTER


- 9.1 The IPv4 header
- 9.2 MTU, MSS and why fragmentation exists
- 9.3 Fragmentation
- 9.4 ICMP — Internet Control Message Protocol
- 9.5 ARP — Address Resolution Protocol


*(Recurring since 2024: header facts, fragmentation numericals, ARP/ICMP statements.)*

## 9.1 The IPv4 header

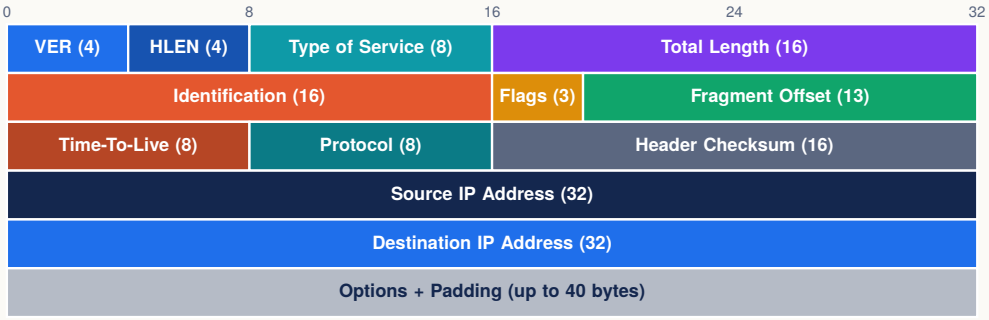



**Figure 9.1** The IPv4 header (20 to 60 bytes), drawn on a 32-bit grid.

| Field | Bits | Meaning / GATE facts |
|---|---|---|
| **VER** | 4 | version: only **IPv4** and **IPv6** became popular |
| **HLEN** | 4 | **header length in units of 4 bytes**. Header size 20–60 B → HLEN value **5 to 15**. (4 bits can't hold 60, so we divide by 4.) **Header bytes =** HLEN × 4 |
| **Type of service** | 8 | priority/QoS (RFC 791; now DSCP/ECN) |
| **Total Length** | 16 | **(Header + Data) in bytes**; range **20 to 65,535**. Data = Total Length − HLEN×4 |
| **Identification** | 16 | same value copied to **all fragments** of one datagram |
| **Flags** | 3 | Reserved, **DF** (Do Not Fragment), **MF** (More Fragments) |
| **Fragment offset** | 13 | position of the fragment's first byte ÷ **8** |
| **TTL** | 8 | maximum hops. **Each router decrements by 1**; at 0 the router **discards** the packet (sends ICMP Time Exceeded). Prevents loops. |
| **Protocol** | 8 | which upper-layer protocol: **TCP = 6, UDP = 17, ICMP = 1** (OSPF = 89) |
| **Header checksum** | 16 | checksum of **header only** |
| **Source / Destination** | 32 + 32 | IP addresses |
| **Options & padding** | ≤ 40 B | source routing, record route, timestamp, padding to a multiple of 4 |

FROM YOUR NOTES

**TTL facts (notes).** *Number of hops a packet can jump before reaching its destination. Prevents network loops and mis-routing. Each hop decrements.* A packet that **arrives at its destination with TTL 0 is accepted by the destination**; an **intermediate router discards** a packet when TTL becomes 0.

FROM YOUR NOTES

**Header checksum facts.** Stores the checksum of the IP header. **Must be re-calculated at every router** (because TTL changes, and when fragmenting). **It is not a security measure** — IP is an **unreliable** protocol. Calculation: header split into **16-bit integers and added** (checksum field treated as **all 0s**), the sum is **1's-complemented** and stored (Ch 4.8).

FROM YOUR NOTES

**Options & padding:** used for **network testing/debugging** — source routing, record route — or just padding.

### ★ "Which header fields can a router change?"

| Always changed by every router | TTL, Header checksum |
|---|---|
| Changed **only when fragmenting** | Total length, Flags (MF), Fragment offset, (HLEN/Options if option copied) |
| Never changed (unless NAT) | Source & Destination address, Protocol, Identification, Version |

TRAP

**NAT** modifies source IP (and L4 ports) — one more reason the IP header checksum must be recomputed.

### IPv6 in 5 lines (for comparison questions)

- **128-bit** addresses; **fixed 40-byte header**; **no header checksum**; **no fragmentation by routers** (**only the source fragments**, notes); no options in the base header (extension headers instead); Hop Limit replaces TTL.

## 9.2 MTU, MSS and why fragmentation exists ★

An IP packet can be up to **65,535 B**, but the **data-link protocol limits how much a frame can carry**. **Ethernet's limit is 1500 B of data.**

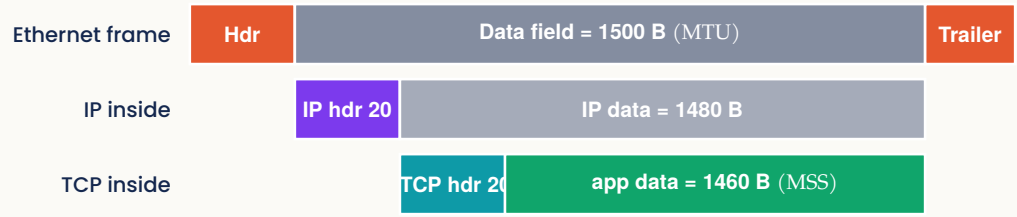



**Figure 9.2** MTU, IP data and MSS — who fits inside whom on an Ethernet link.

- **MTU (Maximum Transmission Unit)** = max data a DL frame can carry (**Ethernet: 1500 B**) — (*your notes: "amount of data allowed at maximum in a DL-layer frame, without headers"*).
- **MSS (Maximum Segment Size)** = max **application data** in one TCP segment = **MTU − IP header − TCP header = 1500 − 20 − 20 = 1460 B**.

TRAP

**Never confuse:** MTU includes the **IP header**; MSS excludes IP and TCP headers.

## 9.3 Fragmentation ★★

If the IP datagram (header + data) is **larger than the MTU of the next link**, the data is **partitioned into fragments**.

Important facts ( notes): - Data-link protocols are **hop-to-hop**, so the MTU **may change between links** ⇒ fragmentation can be done by the **source as well as by intermediate routers**. - **IPv6: only the source fragments.** - Source-side fragmentation in IPv4 can be avoided if the transport layer checks the link's MTU and adjusts its MSS — **but this doesn't prevent fragmentation at intermediate hops.** - **Reassembly happens only at the destination** (fragments may take different paths). - If **any fragment is lost**, the destination eventually **discards all received fragments** after a timeout (ICMP Time-Exceeded is sent) — the whole datagram is lost.

### Header fields used

1. **Identification** — copied into every fragment so the destination groups fragments with the same ID.
2. **Flags**

- **MF (More Fragments): 1 for all fragments except the last**; the **last fragment has MF = 0**.
- **DF (Do not Fragment):** if 1, a router whose MTU is smaller than the packet **drops it** (instead of fragmenting) and sends an **ICMP "fragmentation needed"** message to the source (used for *Path MTU discovery*).
- one Reserved bit.

3. **Fragment offset** — sequences the fragments: **offset = (index of first data byte in the fragment) ÷ 8**.

## Why ÷ 8 ?

Offset field = **13 bits** but Total Length = **16 bits**. Dividing by 8 ($2^3$) lets 13 bits address $2^{13} \times 8 = 2^{16}$ bytes. Consequence:

KEY IDEA

**Every fragment's data size (except the last) must be a multiple of 8 bytes.**
So **(MTU − IP header) must be rounded DOWN to a multiple of 8.**

## Fragmentation recipe (4 steps)

1. **Max data per fragment D = ⌊(MTU − header)/8⌋ × 8.**
2. **Number of fragments = ⌈total data / D⌉.**
3. Fragment *i* (starting at 0) carries bytes [i·D … ]; **offset = start byte / 8**; **MF = 1** except for the last; **Total length = data in that fragment + header.**
4. Last fragment has **data = remainder**. (When re-fragmenting a fragment, the new offset = old offset + (start within fragment)/8 and the **last sub-fragment keeps the original MF** — if the original fragment had MF = 1, *all* sub-fragments have MF = 1.)

## Example 1 (notes) — data 5000 B, MTU 1500, header 20 B

D = 1500 − 20 = 1480 (multiple of 8 ✔ 1480/8 = 185).

| Data bytes | Id | Offset | MF | Total length |
|---|---|---|---|---|
| 0 – 1479 | 10 | 0 | 1 | 1500 |
| 1480 – 2959 | 10 | 185 | 1 | 1500 |
| 2960 – 4439 | 10 | 370 | 1 | 1500 |
| 4440 – 4999 | 10 | 555 | **0** | 580 (560 + 20) |

Offsets: 0/8 = 0; 1480/8 = 185; 2960/8 = 370; 4440/8 = 555 ✔.

**Re-fragmentation (notes): the second fragment (bytes 1480–2959, 1480 B) now meets a link with MTU 800.** - Raw capacity 800 − 20 = 780 B, but **780 is not a multiple of 8** → **use 776** (776/8 = 97). - Sub-fragments: **1480–2255** (776 B, offset 1480/8 = **185**) and **2256–2959** (704 B, offset 2256/8 = **282**). Both keep **MF = 1** (the original fragment wasn't the last).

## Example 2 (notes) — MTU 576, IP header 20, data 1484, Id 101

▶ Max data = 576 − 20 = 556; 556 mod 8 = 4 ≠ 0 → use **552**.

| Data bytes | Frag. offset | MF | Id | Total length |
|---|---|---|---|---|
| 0 – 551 | 0 | 1 | 101 | 572 |
| 552 – 1103 | 69 | 1 | 101 | 572 |
| 1104 – 1483 | 138 | 0 | 101 | 400 |

(552/8 = 69, 1104/8 = 138; last: 380 + 20 = 400.)

## Extra practice

**F1.** Datagram = 20 B header + 4000 B data, MTU 1500, then a later link with MTU 700. - Link 1: D = 1480 → fragments 1480, 1480, 1040 → offsets 0, 185, 370; MF 1, 1, 0. - Link 2: D = $\lfloor 680/8 \rfloor \times 8 = 680$. Fragment 1 (1480) → 680 + 680 + 120 → offsets 0, 85, 170 (MF 1,1,1). Fragment 2 (1480) → 3 pieces likewise. Fragment 3 (1040, MF = 0) → 680 + 360 → offsets 370 and 455, MF = 1, **0**; last total length 380. - **Total fragments reaching the destination = 3 + 3 + 2 = 8.**

**F2.** A datagram has total length 5000 B (header 20 B), MTU 1000. Data = 4980; D = $\lfloor 980/8 \rfloor \times 8 = 976$. Fragments = $\lceil 4980/976 \rceil$ = **6**; offsets 0,122,244,366,488,610; **last fragment carries 4980 − 5×976 = 100 B** (total length 120).

**F3.** A fragment has **offset = 100, MF = 0, total length 200, header 20**. Data in the fragment = 180. It starts at byte $100 \times 8 = 800$. **Original datagram data = 800 + 180 = 980 bytes.** (MF = 0 ⇒ last fragment.)

**F4.** *Why is the maximum IP packet that can be reassembled ≈ 65,535 B?* Total Length is 16 bits (and the offset field covers $2^{13} \times 8 = 65{,}536$ bytes).

## 9.4 ICMP — Internet Control Message Protocol ★

**IP is unreliable.** ICMP is an effort to provide *some* (not complete) reliability by **reporting errors** and **querying network status**.

ICMP tries to answer: 1. What happens if a router must **discard a datagram** because (a) it **can't find a route** to the destination, or (b) the **TTL has reached 0**? 2. What happens if the **destination must discard all received fragments** because it didn't receive all of them **within the time limit**?

Facts: - **ICMP reports errors to the source host only.** - ICMP messages are carried **inside an IP datagram** (Protocol = 1), which is inside a frame: `Frame H | IP H | ICMP msg | Trailer`. - ICMP also lets you check the health of nodes using **query messages**.

### Message format (error reporting)

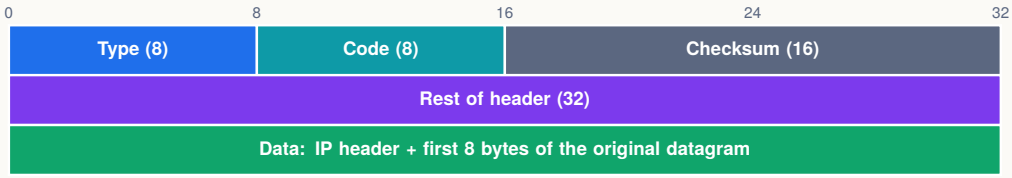



**Figure 9.3** ICMP message format.

### Categories

| Error-reporting messages | Query messages |
|---|---|
| 1. **Destination unreachable** (Type 3) | 1. **Echo request / reply** (Type 8 / 0) — check reachability of a hop (**ping**) |
| 2. **Source quench** (Type 4; flow-control hint, obsolete) | 2. **Timestamp request / reply** — measure **RTT** |

| Error-reporting messages | Query messages |
|---|---|
| 3. **Time exceeded** (Type 11 — TTL = 0 or reassembly timeout) | |
| 4. **Parameter problem** (Type 12) | |
| 5. **Redirection** (Type 5 — "use a better router") | |

### ★ Rules: when is NO ICMP error message generated?

1. **No ICMP error message in response to a datagram carrying an ICMP error message** (prevents infinite error storms).
2. **No ICMP error message for a fragment datagram that is not the first fragment.**
3. **No ICMP error for a datagram with a multicast address, 127.0.0.1 (loopback) or 0.0.0.0.** (also none for broadcast destinations.)

EXAM SHORTCUT

**Tools:** *ping* = ICMP echo. *traceroute* = sends packets with **TTL = 1, 2, 3 ...**; each router that discards a packet replies with **ICMP Time Exceeded**, revealing the path.

## 9.5 ARP — Address Resolution Protocol ★

**Problem:** the data-link layer needs the **MAC address** of the next node, but the network layer knows only the **IP address**.

KEY IDEA

**ARP helps a host within a LAN find the hardware (MAC) address using the IP address.**

FROM YOUR NOTES

**Working.** Four hosts A (10.1), B (10.2), D (10.3), C (10.4) on a LAN. A wants D's MAC: 1. A **broadcasts** an ARP **request** ("who has IP 10.3?") to everyone on the LAN. 2. Everyone gets it; only **D recognises its own IP** and **unicasts an ARP reply** with its MAC address back to A. 3. A stores the mapping in its **ARP cache** (entries expire after a while).

## ARP packet

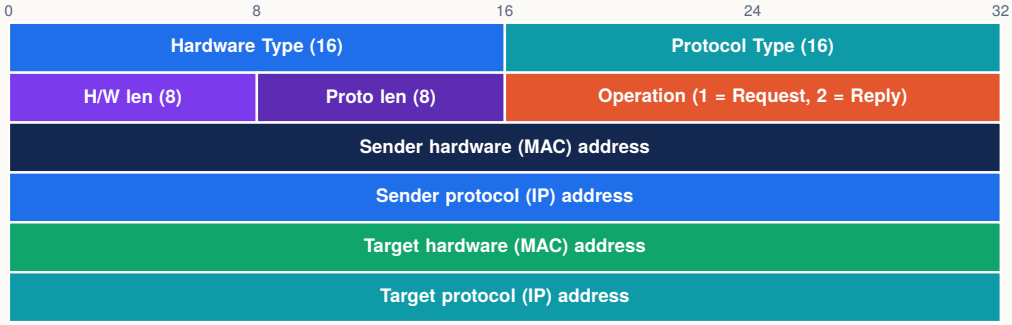



**Figure 9.4** ARP packet format.

- In a **request** the **target hardware address is unknown** → field filled with **0s** (the Ethernet frame itself is sent to the **broadcast address FF:FF:FF:FF:FF:FF = all 1s**).
- In the **reply** the target host fills in its hardware address.
- **An ARP packet is put directly as data in the DL frame** (Ethernet Type 0x0806) — *not* inside an IP datagram.

TRAP

Your notes say "target H/W address is set to all 1s" — the *frame's destination* is all 1s (broadcast); the **ARP request's target-hardware field is all 0s** in the textbook (Forouzan). If a GATE option says "all 0s in the request", that's the standard.

## The four use-cases of ARP ★

| Case | Sender | Destination | What ARP resolves |
|---|---|---|---|
| 1 | **Host** | host in the **same** network | **destination host's** MAC |
| 2 | **Host** | host in a **different** network | look in routing table → **IP of next hop (router)** → ARP for **router's MAC** |
| 3 | **Router** (forwarding a packet) | host in **another** network | same as 2 – ARP for **next-hop router's** MAC |
| 4 | **Router** | host in the **same** network (final hop) | same as 1 – ARP for **destination host's** MAC |

EXAM SHORTCUT

**Rule:** *ARP always resolves the MAC of the* ***next hop****, never of a far-away destination.* **Source/destination IPs stay the same end-to-end; MAC addresses change at every hop.**

**Related:** RARP (MAC → IP, obsolete, replaced by DHCP); **Proxy ARP** (router answers ARP on behalf of hosts).

CHAPTER

# 10

# Routing: Distance Vector, Link State and Real Protocols

Exam priority: ★★ Tier 2

IN THIS CHAPTER


- 10.1 Routing vs forwarding
- 10.2 Distance Vector Routing (DVR)
- 10.3 Count-to-Infinity problem
- 10.4 Link State Routing (LSR) *(notes: “Pending” — completed here)*
- 10.5 Real routing protocols (1-mark matching)


## 10.1 Routing vs forwarding

- **Forwarding** = moving a packet to the right interface using the table (Ch 8).
- **Routing protocols** = **protocols used to populate the routing table** of every router in a network; this information is then used for forwarding.

**Model:** the network is a **graph** — routers are **vertices**, links are **weighted edges**. Weights (costs) can be **hop count, propagation time, delay, bandwidth**, etc.

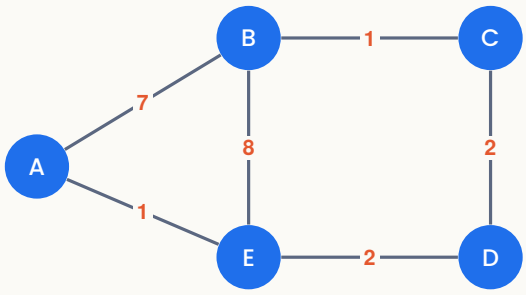



**Figure 10.1** The example network: routers are vertices, link costs are edge weights.

Two popular algorithms: **(1) Distance Vector Routing, (2) Link State Routing.**

## 10.2 Distance Vector Routing (DVR) ★★

KEY IDEA

**Distance vector** = a table at each router mapping **destination router** → **distance (cost)**. Based on the **Bellman-Ford algorithm**.

**Algorithm** 1. **Initially** each router knows only itself (cost 0) and its **direct neighbours** (link cost); everything else = ∞. 2. Each router **periodically shares its distance vector with its neighbours** (only neighbours!). 3. On receiving a neighbour v's vector, router x updates for every destination y: **> D_x(y) = min over neighbours v of [ cost(x, v) + D_v(y) ]** 4. Repeat sharing/updating until **no router changes (convergence)**.

**Note (notes):** *distance vectors* ***don't carry next-hop info*** — the next hop is the neighbour v that gave the minimum (the router remembers it locally).

### Example 1 (notes) — graph A–B 7, A–E 1, B–E 8, B–C 1, C–D 2, D–E 2

Initial vectors:

| A | | B | |
|---|---|---|---|
| A | 0 | A | 7 |
| B | 7 | B | 0 |
| C | ∞ | C | 1 |
| D | ∞ | D | ∞ |
| E | 1 | E | 8 |

(and so on for C, D, E)

**Update at D** after receiving the vectors of **E** (A1, B8, C∞, D2, E0) and **C** (A∞, B1, C0, D2, E∞). D's costs: D–E = 2, D–C = 2. - D→A = min(2 + 1 via E = **3**, 2 + ∞ via C) = **3, next hop E** - D→B = min(2 + 8 via E = 10, 2 + 1 via C = **3**) = **3, next hop C** - D→C = **2** (next hop C); D→E = **2** (next hop E)

| D's new table | Cost | Next hop |
|---|---|---|
| A | 3 | E |
| B | 3 | C |
| C | 2 | C |
| D | 0 | – |
| E | 2 | E |

(*After full convergence, the true shortest costs are: A→B = 6 via E (1+2+2+1), A→C = 5, A→D = 3; B→E = 5 via C; so some entries keep improving in later rounds — good news spreads gradually.*)

## Example 2 (notes) — graph A–B 1, A–C 3, A–D 7, B–C 1, C–D 2

**Initial:** A: (A0, B1, C3, D7) • B: (A1, B0, C1, D∞) • C: (A3, B1, C0, D2) • D: (A7, B∞, C2, D0)

**Round 1** (each uses neighbours' initial vectors):

| For A | cost | next | For B | cost | next | For C | cost | next | For D | cost | next |
|---|---|---|---|---|---|---|---|---|---|---|---|
| A | 0 | – | A | 1 | A | A | 2 | B | A | 5 | C |
| B | 1 | B | B | 0 | – | B | 1 | B | B | 3 | C |
| C | 2 | B | C | 1 | C | C | 0 | – | C | 2 | C |
| D | 5 | C | D | 3 | C | D | 2 | D | D | 0 | – |

**Round 2:** A: D = 4 (via B, because B now knows D = 3: 1 + 3 = 4); others unchanged. **Round 3:** no change → **converged**.

Final: A: B1, C2 (via B), D4 (via B) • B: A1, C1, D3 (via C) • C: A2 (via B), B1, D2 • D: A4 (via C), B3 (via C), C2.

EXAM SHORTCUT

**Rounds to converge ≈ (hop-count of the longest shortest path) + 1.** Each round information travels **one hop**.

### Extra example (link costs): A–B 2, B–C 3, C–D 1, A–D 10

| From → | A | B | C | D |
|---|---|---|---|---|
| A | 0 | 2 | **5** (via B) | **6** (via B) — direct is 10 |
| B | 2 | 0 | 3 | **4** (via C) |
| C | 5 | 3 | 0 | 1 |
| D | **6** (via C) | 4 | 1 | 0 |

## 10.3 Count-to-Infinity problem ★★

KEY IDEA

**"Good news travels fast, bad news travels slow."**

FROM YOUR NOTES

**Notes setup.** Initially only B–C exists: distances to A (for A, B, C) = (0, ∞, ∞). Link A–B is established (cost 1): - Round 1: (0, 1, ∞); Round 2: (0, 1, 2) → **good news (a new link) spread in 2 rounds.** ✔

Now **link A–B goes down.** B and C only exchange vectors with each other. Distance to A:

| Round | B | C | How |
|---|---|---|---|
| before | 1 | 2 | |
| R1 | **3** | **2** | B = 1 + C's 2 = 3 ; C = 1 + B's old 1 = 2 |
| R2 | 3 | **4** | B = 1 + 2 = 3 ; C = 1 + B's 3 = 4 |
| R3 | **5** | 4 | B = 1 + 4 ; C = 1 + 3 |
| R4 | 5 | 6 | … |
| … | | | keeps growing until the cost reaches ∞ |

Why? C believes "I can reach A through B (cost 2)" and B believes "I can reach A through C" — **each depends on the other** (a routing loop). The values climb **very slowly** to infinity.

### Solutions (★ remember all)

| Technique | Idea |
|---|---|
| **Define a small "infinity"** | RIP uses **16 = unreachable**, so counting stops quickly |

| Technique | Idea |
|---|---|
| **Split horizon** | **don't advertise a route back on the interface it was learned from.** (*notes: "C while sharing its DV also shares next-hop info; B seeing that C goes to A through B only doesn't use that info"*) |
| **Poison reverse** | advertise the route back to the neighbour **with cost =** $\infty$ |
| Hold-down timers, triggered updates | wait / send immediately on change |

TRAP

Split horizon **fails for some loops involving 3+ routers**.

## 10.4 Link State Routing (LSR) ★ *(notes: "Pending" — completed here)*

KEY IDEA

**Every router learns the complete map (topology) of the network and computes shortest paths itself using Dijkstra's algorithm.**

**Steps** 1. **Discover neighbours** (HELLO packets) and measure link costs. 2. Build a **Link State Packet (LSP/LSA)**: "I am X; my neighbours are … with costs …". 3. **Flood** the LSP to **all** routers (each router forwards new LSPs on all interfaces except the one it came from; sequence numbers stop loops). 4. Every router now has the **same link-state database** (full graph). 5. Each runs **Dijkstra** to get the shortest-path tree → routing table.

GATE-STYLE QUESTION

**Dijkstra example** — links A–B 1, A–C 4, B–C 2, B–D 5, C–D 1. Source A:

| Step | Pick (min dist, unvisited) | dist(B) | dist(C) | dist(D) |
|---|---|---|---|---|
| init | – | $\infty$ | $\infty$ | $\infty$ |
| A | A (0) | 1 | 4 | $\infty$ |
| B | B (1) | – | min(4, 1+2) = **3** | 1+5 = 6 |
| C | C (3) | – | – | min(6, 3+1) = **4** |
| D | D (4) | | | |

Shortest paths from A: B = 1, C = 3 (via B), D = 4 (via B, C). **Next hop from A for C and D = B.**

**Complexity:** Dijkstra with a binary heap $O((V + E) \log V)$; with an array $O(V^2)$.

### DV vs LS ★

| | Distance Vector | Link State |
|---|---|---|
| Information a router knows | only **distances via neighbours** | **entire topology** |
| Information shared | **whole table → only neighbours** | **only its own links → everyone (flooding)** |
| Algorithm | Bellman-Ford | Dijkstra |
| Convergence | slow; **count-to-infinity** | fast, no loops |
| Message overhead | small per message, periodic | large floods, but only on change |
| Examples | **RIP** | **OSPF** |

## 10.5 Real routing protocols ★ (1-mark matching)

| Protocol | Type | Scope | Metric / Key facts | Transport |
|---|---|---|---|---|
| **RIP** | **Distance vector** | **Intra-AS** (interior) | **hop count; max 15, 16 = ∞**; updates every 30 s | **UDP port 520** |
| **OSPF** | **Link state** | **Intra-AS** | cost (bandwidth-based); **areas** (backbone area 0); Dijkstra | runs **directly over IP** (**protocol 89**) |
| **BGP** | **Path vector** | **Inter-AS** (between ISPs) | AS-path + policies; the glue of the Internet | **TCP port 179** |

- **Autonomous System (AS)** = a network under one administrative control. **Intra-AS** routing = IGP (RIP, OSPF). **Inter-AS** = EGP (BGP).
- **Path vector** = like DV but advertises the **entire AS path** → avoids loops.

CHAPTER

# 11

# Transport Layer — Ports, UDP and TCP Basics

Exam priority: ★★★ Tier 1 — sure-shot

IN THIS CHAPTER


- 11.1 What does the transport layer do?
- 11.2 UDP — User Datagram Protocol
- 11.3 TCP — Transmission Control Protocol
- 11.4 Sequence numbers and acknowledgement numbers
- 11.5 Windows in TCP


## 11.1 What does the transport layer do?

- Required for **process-to-process communication**. IP only gets data to the right *host*; the transport layer gets it to the right *program* on that host.
- It **provides service to the Application layer** and **receives service from the Network layer**.
- **Responsibilities:** (1) **Reliability & flow control**, (2) **Congestion control**.

### Port numbers

To uniquely identify a process on a host, **16-bit port numbers** are used → range **0 to $2^{16} - 1 = 65{,}535$**.

| Range | Name | Use |
|---|---|---|
| **0 – 1023** | **Well-known (system) ports** | reserved for **standard applications** such as HTTP, FTP ... |
| 1024 – 49151 | Registered ports | registered applications |
| 49152 – 65535 | Dynamic / **ephemeral** ports | temporary ports picked by clients (*your notes: "1024 to $2^{16}$−1 are ephemeral"*) |

**Socket address = (IP address, port)**. A connection is identified by the **4-tuple (src IP, src port, dst IP, dst port)**.

Common ports: see the master table in **Chapter 14.8**.

## 11.2 UDP — User Datagram Protocol ★

KEY IDEA

**UDP is a connectionless, unreliable protocol for simple application-to-application communication.**

- **No handshake, no ACK, no retransmission, no flow/congestion control, no ordering.**
- Very low overhead and delay → used by **DNS, DHCP, SNMP, RIP, TFTP, VoIP, video streaming, online games**.

### UDP header — always 8 bytes

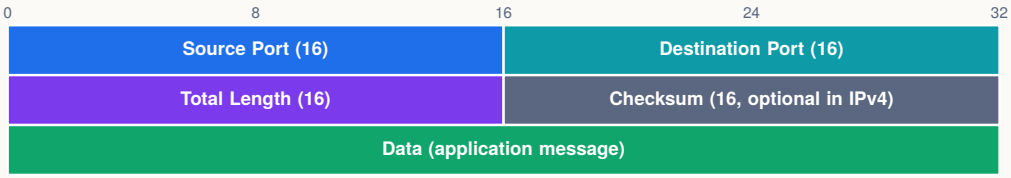



**Figure 11.1** The UDP header: only 8 bytes.

- **Header is always 8 bytes → Total length ranges 8 to $2^{16} - 1 = 65{,}535$ B** (header + data).
- Max UDP **data** = 65,535 − 8 = 65,527 B in principle; with a 20-B IP header, **65,507 B** can actually be carried.

- Checksum covers **UDP header + data + a pseudo-header** (src IP, dst IP, protocol, UDP length). If not used, set to 0 (IPv4 only).

GATE-STYLE QUESTION

*UDP "Total length" field = 1000 → data = 1000 − 8 =* ***992 bytes****.*

## 11.3 TCP — Transmission Control Protocol ★★★

TCP is **full-duplex, connection-oriented, reliable, byte-stream** protocol. - **Full duplex:** both parties send and receive at the same time. - **Connection-oriented:** a logical connection is set up first. - **Byte-oriented:** *each byte has an id* (*sequence number*); TCP logically streams bytes from one process to another. The application hands over a stream; TCP cuts it into **segments of variable size**. - **Reliable:** uses **acknowledgements**, retransmission, checksums. - **Three phases:** (1) **connection establishment**, (2) **data exchange in both directions**, (3) **connection termination**. - Every segment gets a **TCP header (20–60 B)**.

### TCP header

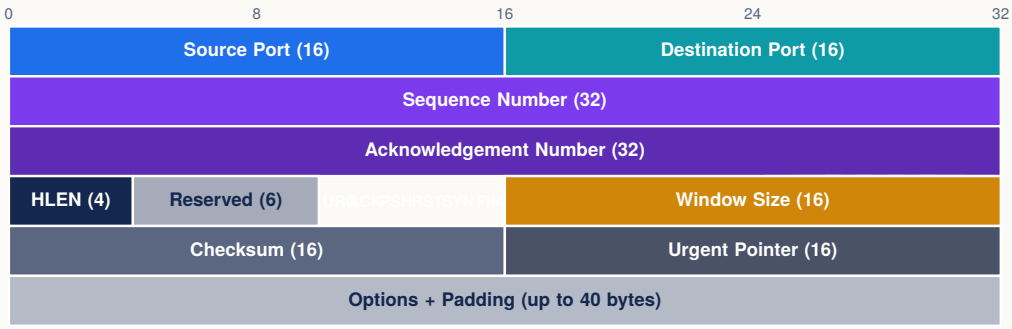



**Figure 11.2** The TCP header (20 to 60 bytes) with its six control flags.

- **Header size: 20 B to 60 B**; **HLEN** field (4 bits) = header length ÷ 4 → values **5 to 15**. (*Header bytes* = HLEN × 4.)

| Field | Meaning |
|---|---|
| **Source port / Dest. port** | uniquely identify the **sending / receiving process** on a host |
| **Sequence number (32)** | sequence number of the **first data byte** in this segment |

| Field | Meaning |
|---|---|
| **Acknowledgement number (32)** | **index of the next byte expected** by the receiver. Acks are **cumulative** and can be **piggy-backed** with data |
| **HLEN** | header length ÷ 4 (20→5, 60→15) |
| **Window size (rwnd)** | the receiver's **current free buffer** in bytes (flow control). Normally 16 bits; the options field can **extend it by 14 bits** → **30 bits total** (window scaling) |
| **Checksum** | covers **TCP header + data + IP pseudo-header fields (src/dst IP ...)** — mandatory |
| **Urgent pointer** | valid only if URG = 1 |
| **Options** | MSS, window scale, SACK, timestamp ... |

### The six flags ★

| Flag | When set |
|---|---|
| **ACK** | the **acknowledgement number is valid** (not garbage). Set in every segment except the very first SYN |
| **SYN** | **synchronisation** — set only in the **first packet each side sends** (connection setup) |
| **FIN** | set in the **last packet** — the sender wants to **end the connection** |
| **RST** | **reset** the connection (abort/refuse) |
| **PSH** | **push**: normally the receiving TCP waits for the process to pull data; if the sender wants the data pushed to the application **immediately**, it sets PSH |
| **URG** | **urgent pointer is valid**: data from the segment's start up to the pointed byte is high-priority |

## 11.4 Sequence numbers and acknowledgement numbers ★★★

### Sequence numbers

TCP **numbers every byte**. After segments are created they get 32-bit sequence numbers: 1. The **first segment** gets a **random Initial Sequence Number (ISN)** in the range **0 to** $2^{32} - 1$. 2. **Seq(next segment) = Seq(previous segment) + number of bytes in the previous segment.**

FROM YOUR NOTES

**Example (notes).** 500 bytes (byte 0–499) split into segments of bytes **0–149, 150–299, 300–449, 450–499** and ISN = 2152.

| Segment | Bytes | Seq. No. |
|---|---|---|
| 1 | 0 – 149 | **2152** (random) |
| 2 | 150 – 299 | 2152 + 150 = **2302** |
| 3 | 300 – 449 | 2302 + 150 = **2452** |
| 4 | 450 – 499 | 2452 + 150 = **2602** |

### Acknowledgement number

KEY IDEA

**Ack number = number (index) of the next byte the receiver expects.**

If the first byte of a transfer has sequence number **x** and the receiver gets 500 bytes (x ... x+499) → it sends **ACK = x + 500**. In general **Ack = initial seq + number of bytes received in order**.

EXAM SHORTCUT

**Segment of L bytes with seq = s → its ack = s + L.** The sender's next segment's seq = s + L.

TRAP

Seq number / ack number are **byte numbers, not segment numbers** (unlike GBN/SR in the DLL).

## 11.5 Windows in TCP

TCP is full duplex → each side has a **send window** and a **receive window**.

**Send window (sender side):** bytes ... `199 | 200 | 201 … 260 | 261 | 262 …` - Two pointers: **first outstanding (unacknowledged) byte** and **next byte to send**. - Bytes between the two pointers are *sent but not yet acknowledged*. - Window size = min(rwnd, cwnd) (rwnd = flow control, cwnd = congestion control, Ch 13).

**Receive window (receiver side):**

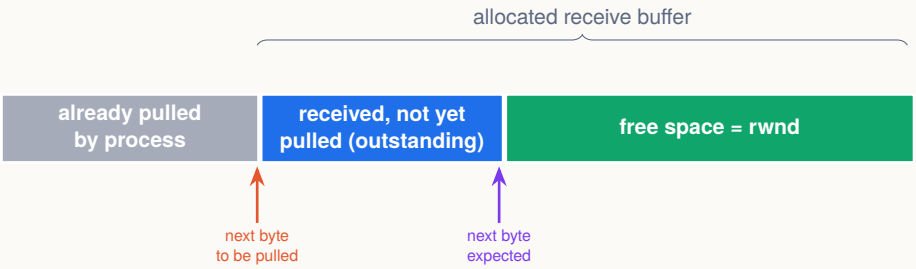



**Figure 11.3** The receiver's buffer: rwnd is the free space that remains.

- The receiver **allocates a buffer of fixed size**. Two pointers: **(a)** received bytes not yet pulled by the process; **(b)** the next byte expected.
- **rwnd = buffer allocated − bytes outstanding (received but not yet pulled by the process).**

CHAPTER

# 12

# TCP Connection, Acknowledgement, Retransmission, Flow Control, Timers

Exam priority: ★★★ Tier 1 — sure-shot

IN THIS CHAPTER


- 12.1 Connection establishment — 3-way handshake
- 12.2 Connection termination
- 12.3 TCP state machine
- 12.4 Acknowledgement policy
- 12.5 Retransmission
- 12.6 TCP timers (4 important)
- 12.7 Flow control in TCP
- 12.8 Sequence-number wrap-around (asked in 2018)


## 12.1 Connection establishment — 3-way handshake ★★★

Although TCP is full duplex, **one party initiates** — the **TCP client**; the one that waits is the **TCP server**.

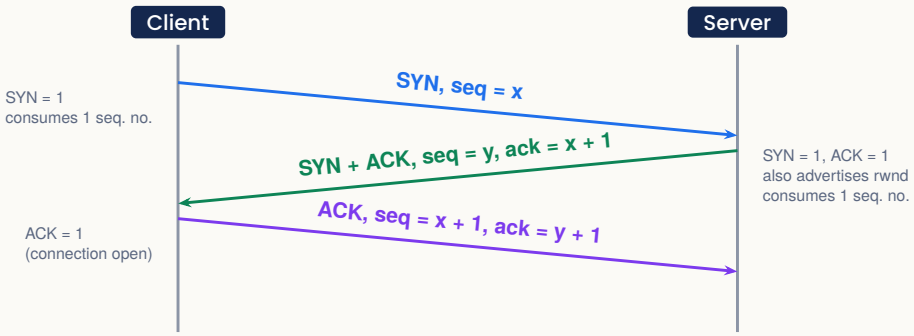



**Figure 12.1** TCP three-way handshake: SYN and SYN+ACK each consume one sequence number.

1. **SYN segment (client → server).** SYN = 1. It is only a *synchronisation* request (server may accept/reject), so it carries **no real data**, but it **consumes one sequence number** (as if a 1-byte dummy data was sent) with **Seq = x**.
2. **SYN + ACK segment (server → client).** ACK = 1 (replying) and SYN = 1 (first packet server → client). **Ack No. = x + 1**. The server also picks its own **Seq = y** and the segment likewise **consumes one sequence number (1 B dummy)**.
3. **ACK segment (client → server).** ACK = 1, **Ack No. = y + 1**. May be **piggy-backed with real application data** (then it **consumes** sequence numbers equal to the data bytes) or be a **pure ACK** (consumes **no** sequence number).

- While accepting/acknowledging the request, **both sides advertise their window size (rwnd)**.

FROM YOUR NOTES

**Table from the notes**

| Segment | SYN | ACK | Seq. No. | Ack No. | RWND | Consumes seq no.? |
|---|---|---|---|---|---|---|
| Client → Server | 1 | 0 | 5000 | – | – | **Yes** (1 B dummy) |
| Server → Client | 1 | 1 | 8000 | 5001 | 3200 | **Yes** (1 B dummy) |
| Client → Server | 0 | 1 | 5001 | 8001 | 2100 | **Yes if data is sent; otherwise no** |

KEY IDEA

**Rule:** *The first two segments consume a sequence number because they have to be acknowledged. The last segment, if it carries no data, needs no acknowledgement and consumes none.*

GATE-STYLE QUESTION

**Practice.** Client ISN = 1000, server ISN = 5000; the 3rd segment carries 200 bytes of data. - SYN: seq = 1000 • SYN+ACK: seq = 5000, ack = 1001 • ACK+data: seq = 1001, ack = 5001, 200 B - Server then acknowledges with **ack = 1201**; client's next segment has **seq = 1201**.

EXAM SHORTCUT

**"What consumes a sequence number?" → SYN (1), FIN (1), and every data byte. A pure ACK consumes none.**

## 12.2 Connection termination ★★

Termination can happen in **two ways: 3-way** or **4-way handshake**.

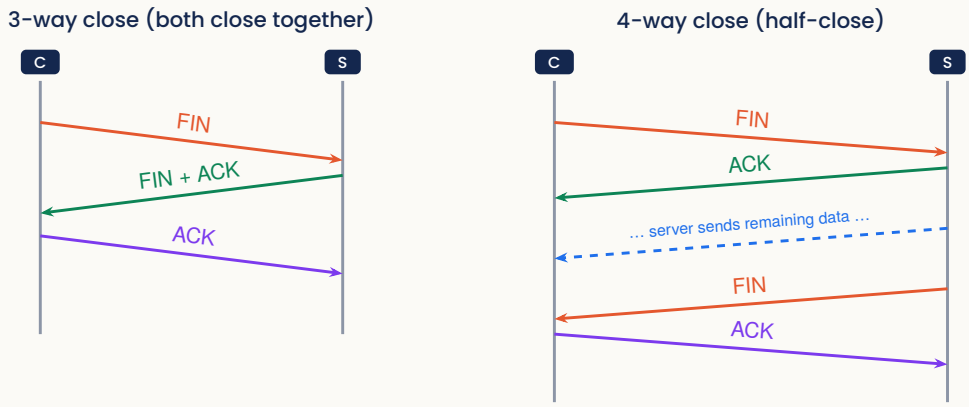



**Figure 12.2** TCP connection termination: 3-way close and 4-way (half-close) close.

- **Client** sends a **FIN** segment (last segment, FIN = 1). It may or may not contain data but **consumes one sequence number**.
- **Server** responds in one of two ways:
  - **It also wants to terminate** → sends **FIN + ACK** (may or may not contain data; consumes one seq. no.) → **3-way handshake**. The client replies with a final **ACK**.

- **It still has data to send** → sends just an **ACK**; transfers the remaining data (server → client); then sends its own **FIN**; the client acks it → **4-way handshake** (this is the **half-close**).

FROM YOUR NOTES

**Sequence-number consumption - 3-way:** FIN and FIN+ACK **always consume** $\geq 1$ sequence number (1 if no data, more if data). The final ACK **may or may not carry data** (per your notes) → it consumes sequence numbers only if it carries data; a pure ACK consumes none. - **4-way:** the two FINs each consume $\geq 1$. The **ACK from S→C** may or may not carry data (so may or may not consume). The **final ACK from C→S can't carry data** (the client already closed) → **consumes none**.

## 12.3 TCP state machine ★

**Client = active closer, Server = passive closer.**

| Side | Transition | Event / action |
|---|---|---|
| Both | **CLOSED** | initial state |
| Server | CLOSED → **LISTEN** | passive open (listen on socket) |
| Client | CLOSED → **SYN_SENT** | sent SYN |
| Server | LISTEN → **SYN_RCVD** | received SYN, sent SYN+ACK |
| Client | SYN_SENT → **ESTABLISHED** | received SYN+ACK, sent ACK |
| Server | SYN_RCVD → **ESTABLISHED** | received ACK |
| both | **ESTABLISHED** | send and receive data |
| **Active closer** | ESTABLISHED → **FIN_WAIT_1** | sent FIN |
| | FIN_WAIT_1 → **FIN_WAIT_2** | received ACK of FIN |
| | FIN_WAIT_2 → **TIME_WAIT** | received FIN, sent ACK |
| | FIN_WAIT_1 → **TIME_WAIT** | received FIN+ACK, sent ACK (3-way) |
| | FIN_WAIT_1 → **CLOSING** | received FIN (simultaneous close), sent ACK; → TIME_WAIT on receiving ACK |

| Side | Transition | Event / action |
|---|---|---|
| | TIME_WAIT → **CLOSED** | after **2 × MSL timeout** |
| **Passive closer** | ESTABLISHED → **CLOSE_WAIT** | received FIN, sent ACK |
| | CLOSE_WAIT → **LAST_ACK** | sent FIN |
| | LAST_ACK → **CLOSED** | received ACK |

- **MSL = Maximum Segment Lifetime.** TIME_WAIT lasts **2·MSL** so that (a) the last ACK, if lost, can be re-sent when the peer retransmits FIN, and (b) old duplicate segments from this connection die out. (*Your notes write "2 MSS timeout" — it is **2·MSL**, not the segment size.*)

## 12.4 Acknowledgement policy ★

TCP ACKs are **cumulative** and can be **piggy-backed**.

When a receiver gets a segment: 1. **If it has data to send → piggy-back** the ACK immediately, without delay. 2. If no data to send back, three options: (a) send ACK immediately, (b) wait for some time, (c) wait for some packets and send a cumulative ACK.

**TCP waits for both: some time (≤ ~500 ms) or 2 segments — whichever happens first** (*delayed ACK*). TCP behaves like **Selective Repeat on the receive side** (it **buffers out-of-order segments**) **but uses cumulative ACKs like Go-Back-N**. If the received segment is **out of order**, it can't cumulatively acknowledge → **it sends an ACK immediately** (a duplicate ACK for the last in-order byte).

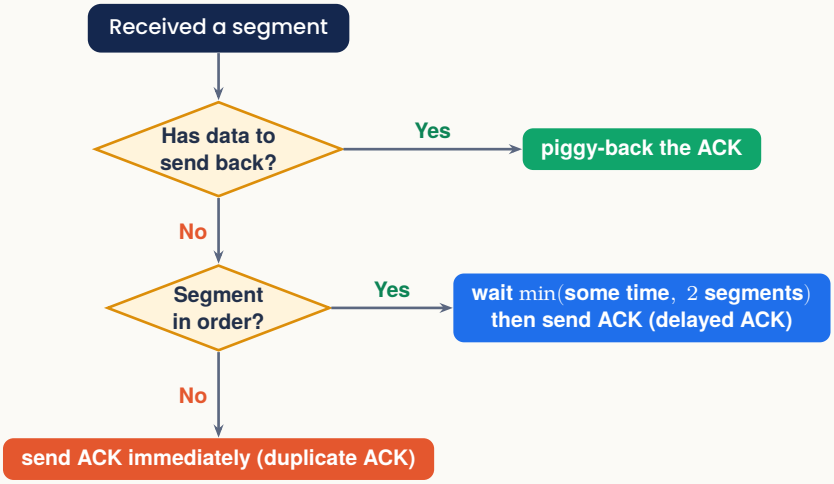



**Figure 12.3** TCP acknowledgement policy as a decision flow.

- A TCP receiver may store out-of-order data in a temporary buffer, but **delivers data to the application in order only.**

## 12.5 Retransmission ★★

Two situations trigger retransmission:

| Trigger | What the sender knows | Meaning |
|---|---|---|
| **1. Timeout** (RTO expires) | no ACK for the lost packet came back | **very high congestion** |
| **2. 3 duplicate ACKs** (**Fast retransmit**) | the segment is lost **but at least 3 later segments were delivered** (they produced the dup-ACKs) | **congestion is high but not as much** |

- Retransmission on timeout = *RTO* wait; on 3 dup-ACKs = **Fast Retransmission** (doesn't wait for the long timeout).

GATE-STYLE QUESTION

*Segments* (*seq*, *len*): (100,50), (150,50), (200,50), (250,50), (300,50). *Segment 150 is lost.* Receiver ACKs: 150 (after 100), then **dup-ACK 150** for 200, 250, 300 → 3 duplicate ACKs → sender **retransmits seq 150 immediately**. When it arrives, receiver (which buffered 200–349) sends **cumulative ACK = 350**.

### Retransmission timeout (RTO) and RTT estimation

- **RTO is set from the measured RTT.**
  - **Measured RTT ($R_m$):** time from sending a segment to receiving its ACK.
  - **Smoothed RTT:** $R_s(0) = R_m(0)$; $R_s(\mathbf{t}) = \alpha \cdot R_m(\mathbf{t}) + (\mathbf{1} - \alpha) \cdot R_s(\mathbf{t-1})$, with $\alpha$ **generally 1/8**.
- GATE-Q $R_s(old) = 100\ ms$, *new sample* $R_m = 180\ ms$, $\alpha = 1/8 \rightarrow R_s(\text{new}) = 0.125 \times 180 + 0.875 \times 100 = 22.5 + 87.5 =$ **110 ms**.
- Modern TCP: **RTO = SRTT + 4 × RTTVAR** (Jacobson). **Karn's rule:** don't take RTT samples from retransmitted segments. **On each timeout RTO is doubled (exponential back-off).**

## 12.6 TCP timers (4 important)

| Timer | Purpose / typical value |
|---|---|
| **Retransmission timeout (RTO) timer** | set based on RTT; if it expires → retransmit |
| **Time-wait timer** | after closing from the client side; **2 × MSL** (generally 2 × 60 s etc.) |
| **Keep-alive timer** | used by **server TCP** to detect a dead idle client; generally **2 hours** |
| **Persistence timer** | used when **rwnd becomes 0** — keeps probing so a lost "window open" ACK doesn't cause deadlock |

## 12.7 Flow control in TCP ★★

KEY IDEA

**Flow control in TCP uses the receiver's window size (rwnd).** The sender decides its send window **based on the advertised rwnd**: **LastByteSent** − **LastByteAcked** $\leq \min(\text{rwnd}, \text{cwnd})$**.**

FROM YOUR NOTES

**Notes example.** Client = sender, Server = receiver. After the handshake: 1. In the **SYN+ACK** the receiver advertises **rwnd = x** bytes. Sender adjusts its window. 2. Sender sends **t bytes** ($\mathbf{t \leq x}$); the receiver now has t bytes

**outstanding (not yet pulled by the application) → new rwnd = x − t**, advertised in the ACK. Sender adjusts again. 3. Suppose between steps 2 and 3 the application **consumes k bytes (k ≤ t) → rwnd = x − t + k** (k bytes freed). ...and so on.

GATE-STYLE QUESTION

**Numeric.** Buffer 5000 B; 3000 B received, application has read 1000 B → outstanding = 2000 → **rwnd = 5000 − 2000 = 3000 B**.

EXAM SHORTCUT

**Throughput limited by window: Throughput ≤ window / RTT.** GATE-Q *rwnd = 32 KB, RTT = 80 ms, link 1 Gbps:* 32 × 1024 × 8 = 262,144 bits / 0.08 s = ≈ **3.28 Mbps** (the link is hardly used). To fill the link you need **window ≥ BDP = BW × RTT**.

- **rwnd = 0** → sender stops; the **persistence timer** sends tiny probe segments.
- **Silly Window Syndrome:** avoided by **Nagle's algorithm (sender)** and **Clark's/delayed-ACK (receiver)**.

## 12.8 Sequence-number wrap-around ★ (asked in 2018)

TCP's sequence space = $2^{32}$ **bytes**. **Wrap-around time = $2^{32}$ × 8 bits / Bandwidth(bps)** — the time to send $2^{32}$ bytes. Must be **> MSL** or an old segment may be mistaken for a new one. - 1 Gbps: $4.295\times10^9 \times 8 = 3.436\times10^{10}$ bits ÷ $10^9$ = **34.4 s** - 10 Gbps: **3.44 s** (reason for **PAWS / timestamps** option) - 10 Mbps: ≈ 3436 s

EXAM SHORTCUT

**Generic:** *n-bit sequence number field, sending B bytes/s → wrap time* = $\frac{2^n}{B}$ *seconds.* (If sequence numbers number **frames** not bytes, divide by frame rate instead.)

CHAPTER

# 13

# TCP Congestion Control

Exam priority: ★★★ Tier 1 — sure-shot

IN THIS CHAPTER


- 13.1 What is congestion?
- 13.2 The congestion window (cwnd)
- 13.3 Phase 1 — Slow Start
- 13.4 Phase 2 — Congestion Avoidance
- 13.5 Reacting to congestion — Tahoe and Reno
- 13.6 TCP vs UDP (1-mark favourite)


## 13.1 What is congestion?

**Congestion = too much traffic in the network.** Causes: packet drops, TTL expiry, link failures, overloaded routers. Result: **sent packets or ACKs are lost**.

KEY IDEA

**IP doesn't handle congestion explicitly → TCP does.**

### How TCP detects congestion (2 heuristics)

1. **Timeout** — a timer is started for every sent packet; **expiry = congestion** (severe).
2. **3 duplicate ACKs** — three ACKs with the same Ack number (mild: later packets are still getting through).

## 13.2 The congestion window (cwnd)

KEY IDEA

**Congestion window (cwnd)** = a theoretical window size adjusted based on the congestion-control **phase** and the **network traffic**. **cwnd is measured in MSS.**

KEY FORMULA

$$W_{\text{send}} = \min(\text{rwnd},\ \text{cwnd})$$

The algorithm has **two phases** and switches between them: 1. **Slow Start (SS) — exponential increase** 2. **Congestion Avoidance (CA) — additive increase**

Assumptions in standard GATE problems (as in the notes): **(1)** every packet size **= 1 MSS** (MSS = MTU − 20 − 20 generally); **(2) every packet is individually acknowledged (1 ACK per packet)**; rwnd is large (not the limit).

## 13.3 Phase 1 — Slow Start

- **cwnd starts at 1 MSS.**
- **On every ACK: cwnd = cwnd + 1 MSS.**
- ⇒ **In one RTT the cwnd doubles** ($1 \to 2 \to 4 \to 8 \to \ldots$): *after every RTT — the time to deliver a packet and receive its acknowledgement — cwnd doubles, assuming 1 ACK per packet.*

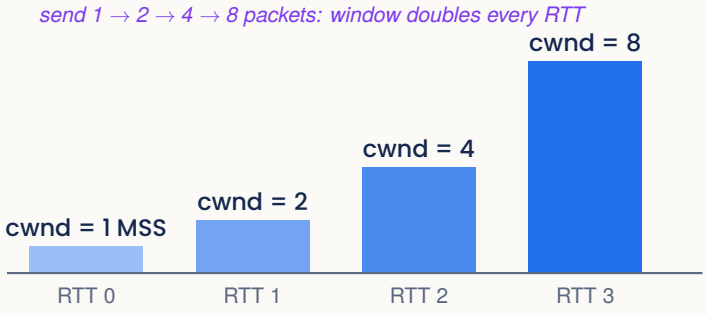



**Figure 13.1** Slow start: cwnd doubles every RTT (one ACK per packet).

TRAP

**Cap rule:** in slow start cwnd **never exceeds ssthresh** — when doubling would cross ssthresh, cwnd is set **equal to ssthresh** and the algorithm moves to CA.

### With delayed ACK (1 ACK for 2 packets)

The growth is slower. Notes example: initial 1 MSS → send 1 packet; receiver times out and ACKs → **2 MSS**; send 2 packets, receiver sends 1 ACK → **3 MSS**; send 3 packets, receiver ACKs two and waits for the third → **4 MSS**; if the receiver times out and ACKs the third → **5 MSS**, else the sender sends another packet and cwnd = 5 MSS only. *"The general assumption while solving problems is 1 ACK per packet."*

## 13.4 Phase 2 — Congestion Avoidance

- cwnd increases **by 1 MSS per RTT** (once ACKs for **all packets of the current window** have arrived).
- Per-ACK form: **cwnd = cwnd + 1/cwnd** (so cwnd ACKs → +1).
- Growth is **linear** (**additive increase**).

### Slow-start threshold (ssthresh)

KEY IDEA

**ssthresh** = the cwnd value which, **when reached or exceeded**, moves us from **Slow Start** → **Congestion Avoidance** (condition: **cwnd** ≥ **ssthresh**).

## 13.5 Reacting to congestion — Tahoe and Reno ★★★

Congestion can occur in **either phase**, due to a **timeout** or **3 duplicate ACKs**. The handling depends on the implementation.

## Implementation 1 — TCP Tahoe (notes' "Taho")

**Does not differentiate between timeout and 3 dup-ACK.** On either event:

KEY FORMULA

$$\text{ssthresh} = \frac{\text{cwnd}}{2}, \qquad \text{cwnd} = 1 \;\Rightarrow\; \text{Slow Start}$$

## Implementation 2 — TCP Reno (with fast recovery, simplified as in the notes)

| Event | ssthresh | cwnd | Next phase |
|---|---|---|---|
| **Timeout** | **cwnd / 2** | **1** | **Slow Start** |
| **3 dup ACKs** | **cwnd / 2** | **cwnd / 2** (= new ssthresh) | **Congestion Avoidance** |

(Full Reno uses ssthresh + 3 during fast recovery and deflates to ssthresh — the exam answers use the simplified form.)

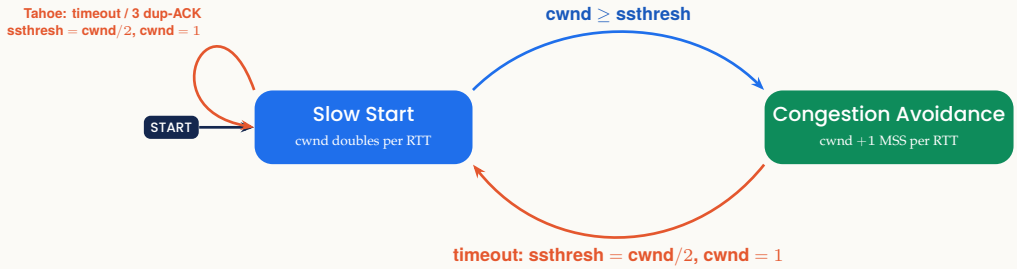



**Figure 13.2** Congestion-control phases and the transitions between them.

## Worked example (notes) — Tahoe

TCP starts data transfer with **ssthresh = 16 MSS**, begins in slow start with cwnd = 1. **Timeout occurs after the 3rd RTT, and again after the 7th RTT.**

| After RTT # | cwnd (MSS) | Phase / event |
|---|---|---|
| 0 | 1 | SS |
| 1 | 2 | SS |

| After RTT # | cwnd (MSS) | Phase / event |
|---|---|---|
| 2 | 4 | SS |
| 3 | **8** | **timeout** → ssthresh = 8/2 = **4**, cwnd = 1, SS |
| 4 | 2 | SS |
| 5 | 4 | cwnd = ssthresh → **SS** → **CA** |
| 6 | 5 | CA (+1) |
| 7 | **6** | **timeout** → ssthresh = 6/2 = **3**, cwnd = 1, SS |
| 8 | 2 | SS |
| 9 | 3 | cwnd = ssthresh (3) → **SS** → **CA** (*2 would double to 4 > 3, so capped at 3*) |
| 10 | 4 | CA |
| 11 | 5 | CA |

Sequence: **1, 2, 4, 8 | 1, 2, 4, 5, 6 | 1, 2, 3, 4, 5, ...** (this is the saw-tooth graph in the notes).

*The notes redo the same example with Implementation 2* (all events are timeouts, so the numbers are the same: ssthresh = 8/2 = 4, then 6/2 = 3).

## Tahoe vs Reno (3 dup-ACK case)

ssthresh = 16, cwnd grows: 1, 2, 4, 8, **16** (=ssthresh → CA), 17, 18, 19, **20** → at cwnd = 20 **3 duplicate ACKs** arrive. - **Tahoe:** ssthresh = **10**, cwnd = **1** → 1, 2, 4, 8, 10, 11, ... - **Reno:** ssthresh = **10**, cwnd = **10** (CA) → 10, 11, 12, ...

## More GATE-style numericals

1. **ssthresh = 8, cwnd = 1, no loss. cwnd after 6 RTTs?** 1,2,4,8 (=ssthresh at RTT 3 → CA) → 9 (RTT 4), 10 (5), **11 (6)**. **Answer: 11 MSS.**
2. **Tahoe, timeout when cwnd = 24. Find cwnd after 5 RTTs.** ssthresh = 12, cwnd = 1 → 2, 4, 8, 12 (RTT 4, reaches ssthresh), **13 (RTT 5)**. **Answer: 13.**
3. **Reno, 3 dup-ACKs at cwnd = 24. cwnd after 3 RTTs?** ssthresh = 12, cwnd = 12 (CA) → 13, 14, **15**. **Answer: 15.**
4. **Largest window before first loss:** if loss happens when cwnd = W, new ssthresh = W/2 ⇒ use **W/2** as the switching point.

### Shortcuts for congestion questions

- **Slow start:** after k RTTs cwnd $= 2^k$ (until it hits ssthresh).
- **Time to reach ssthresh S from 1:** $\lceil \log_2 S \rceil$ RTTs.
- **Congestion avoidance:** after k more RTTs, cwnd = ssthresh + k.
- **Always halve the cwnd *at the moment of loss*, not the cwnd before the last increase.**
- **Window actually used = min(cwnd, rwnd)**; if rwnd < cwnd the sender is limited by the receiver.
- **Throughput ≈ cwnd × MSS / RTT**; AIMD average ≈ **0.75** × $W_{\max}$ **/ RTT**.
- Timeout is worse than 3 dup-ACKs (Tahoe & Reno both reset to 1 on timeout).

## 13.6 TCP vs UDP (1-mark favourite)

| | TCP | UDP |
|---|---|---|
| Connection | connection-oriented | connectionless |
| Reliability | reliable (ACK, retransmission) | unreliable |
| Ordering | in-order delivery | none |
| Flow/congestion control | yes | no |
| Header | 20–60 B | 8 B |
| Unit | byte stream (segments) | message (datagram) |
| Speed/overhead | slower | faster |
| Uses | HTTP, FTP, SMTP | DNS, DHCP, streaming, VoIP |

CHAPTER

# 14

# Application Layer

Exam priority: ★★ Tier 2

IN THIS CHAPTER


- 14.1 Client–server idea
- 14.2 E-mail system
- 14.3 DNS — Domain Name System
- 14.4 HTTP — HyperText Transfer Protocol
- 14.5 FTP — File Transfer Protocol
- 14.6 DHCP — Dynamic Host Configuration Protocol
- 14.7 Other protocols — quick facts
- 14.8 MASTER TABLE — Protocols, ports, transport (memorise!)


*(Mostly 1-mark "protocol roles" questions: learn the facts in this chapter, especially the port table in 14.8.)*

## 14.1 Client–server idea

Application-layer programs run on end hosts. A **client** starts the conversation; a **server** waits. They use the transport layer (TCP or UDP) through **ports**.

## 14.2 E-mail system ★

- E-mail is **asynchronous**: **both users don't need to be online at the same time.**
- **User Agent (UA):** application program that lets a user send/read e-mail (Outlook, Thunderbird, Gmail app).
- **Mail Server (MS):** server that sends and receives mail using various protocols. **UAs may go offline; mail servers never go down**, so they act as **intermediaries** between two user agents.
- The two UAs can be connected to **different mail servers** (e.g. Gmail MS and Yahoo MS).

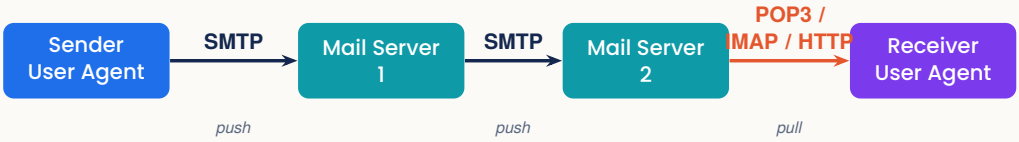



**Figure 14.1** E-mail path: SMTP pushes, POP3/IMAP/HTTP pull.

| Protocol | Purpose | Type | Port (TCP) |
|---|---|---|---|
| **SMTP** (Simple Mail Transfer Protocol) | **Sender UA → its mail server** and **mail server → mail server** | **PUSH** | **25** (submission 587) |
| **POP3** (Post Office Protocol v3) | **mail server → receiver UA**; **moves** mail from MS to UA (downloads & deletes). Obsolete | **PULL** | **110** |
| **IMAP** (Internet Message Access Protocol) | mail server → receiver UA; **copies** mail (mail stays on server; folders; sync across devices) | **PULL** | **143** |

EXAM SHORTCUT

**Memory:** *SMTP pushes (sending side); POP3 & IMAP pull (receiving side).* **SMTP can't be used to retrieve mail.** - SMTP commands: `HELO`, `MAIL FROM`, `RCPT TO`, `DATA`, `QUIT`. Messages are 7-bit ASCII → **MIME** is used for attachments/non-ASCII. Web-mail (Gmail) uses **HTTP** between browser and server.

## 14.3 DNS — Domain Name System ★★

**Problem:** hosts are identified by **IP addresses**, which are hard to remember. People use **hostnames** (www.google.com); **DNS maps hostname ↔ IP address.**

### Services provided by DNS

1. **Hostname → IP translation**
2. **Host aliasing** (a host can have several names; canonical name)
3. **Mail-server aliasing**
4. **Load balancing** (one name → many IPs, rotated)

### Structure: a distributed hierarchy

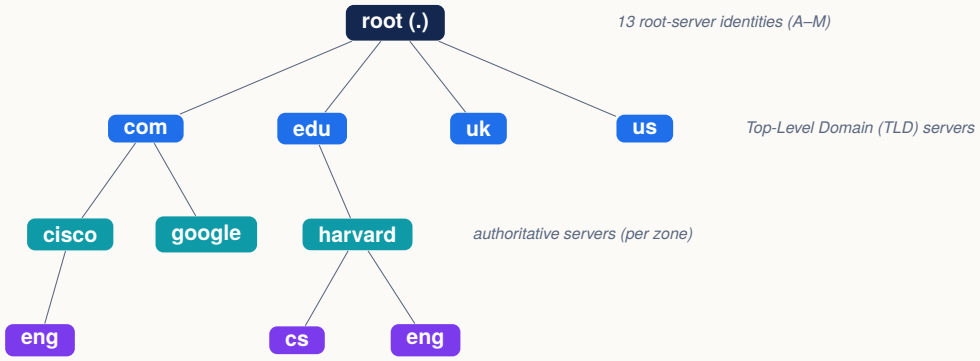



**Figure 14.2** The DNS hierarchy: root, TLD and authoritative servers.

(There are about 250 top-level domains in total, per the notes.)

| Server type | Role |
|---|---|
| **Root DNS servers** | hold the entry of the root node: ⟨**top-level-domain name, TLD server IP**⟩ **pairs. 13 in the world** (logical identities; each is replicated worldwide). |
| **TLD servers** | have the mapping for **top-level domains** (com, edu, org, in, uk ...) → point to authoritative servers |
| **Authoritative DNS servers** | **hold the IP addresses of all hostnames in their zone** (and may hold others). Each **zone** has an authoritative server. |

| Server type | Role |
|---|---|
| **Local DNS server (resolver)** | the DNS server **clients connect to on start-up** (given by ISP/DHCP). If it doesn't know an IP, it **queries the root-level** server (and caches the answers). |

## Two ways of querying ★

Suppose the client wants **A.B.in**.

**(1) Iterative** — *the local DNS does all the work; every server just replies with "ask this server next".*

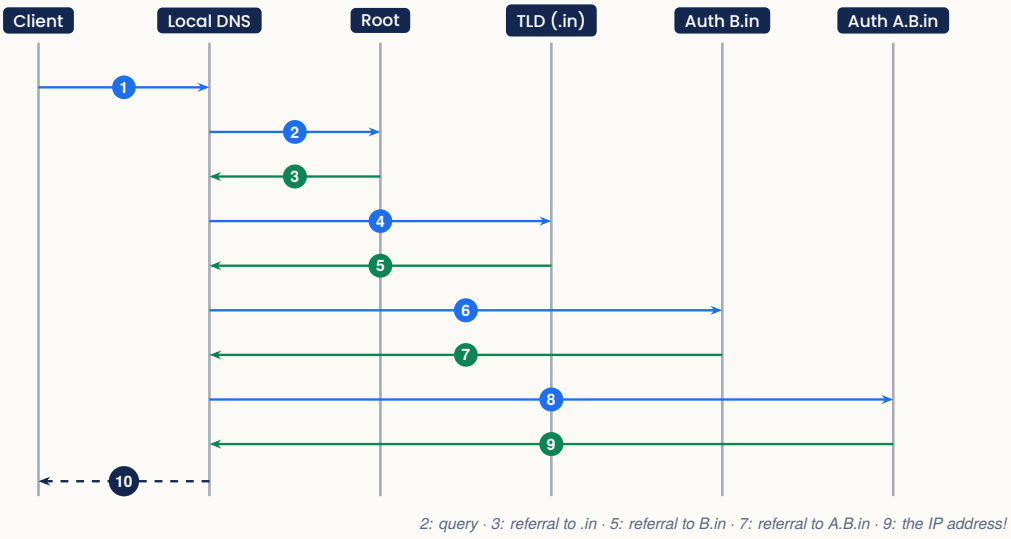



**Figure 14.3** Iterative DNS resolution of A.B.in (10 messages).

**(2) Recursive** — *each server forwards the query to the next one and returns the final answer back along the same path.*

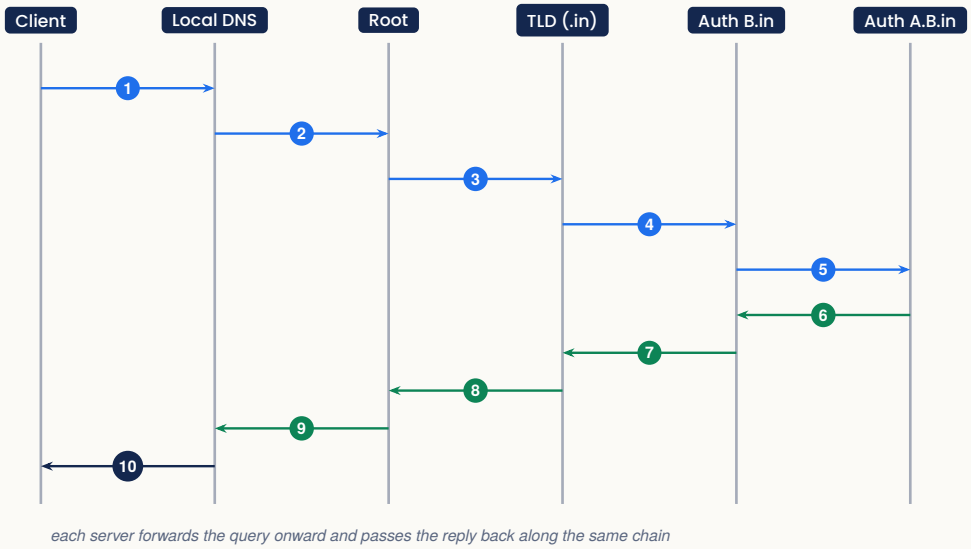



**Figure 14.4** Recursive DNS resolution of A.B.in (10 messages).

Both take **10 messages** in this example; the difference is **who bears the load** (recursive puts load on the higher-level servers; in practice the client→local query is recursive and local→others is iterative).

EXAM SHORTCUT

**Facts:** DNS mostly uses **UDP port 53** (queries $< 512$ B); **TCP port 53** for **zone transfers** and large answers. **Caching** with **TTL** reduces traffic.

## Resource records

| Type | Maps |
|---|---|
| **A** | hostname → IPv4 address |
| **AAAA** | hostname → IPv6 address |
| **NS** | domain → authoritative name server |
| **CNAME** | alias → canonical name |
| **MX** | domain → mail server |
| **PTR** | IP → hostname (reverse lookup) |
| **SOA** | zone information |

## 14.4 HTTP — HyperText Transfer Protocol ★★

- Web protocol; **the server uses TCP port 80** (HTTPS = 443).
- **Stateless protocol** — the server keeps no memory of past requests (cookies are used to add state).
- Request/response: **request line** (method, URL, version) + headers + (body); **response** has a **status line** + headers + body.
  - Methods: **GET** (fetch), **POST** (submit data), **HEAD** (headers only), **PUT**, **DELETE**.
  - Status codes: **200 OK, 301 Moved Permanently, 304 Not Modified, 400 Bad Request, 404 Not Found, 500 Internal Server Error**.

### Two versions

| | HTTP 1.0 — Non-persistent | HTTP 1.1 — Persistent (default) |
|---|---|---|
| TCP connections | **a new TCP connection for each request/response cycle** (data transferred until EOF) | **one TCP connection can carry multiple request/response cycles** |
| Variants | – | **Non-pipelined** and **pipelined** |

**Pipelined requests (notes):** once you get the base page and need **multiple linked files (images, JS, CSS)**, the **requests can be sent in parallel (back-to-back, without waiting for each response)** over the same connection.

### ★ Page-load time calculations

Let **RTT** = round-trip time, ignore transmission time. A page = **1 base HTML + n objects**. - **Non-persistent, serial:** each object needs **1 RTT (TCP handshake) + 1 RTT (request/response) = 2 RTT** $\rightarrow$ **(n + 1)** $\times$ **2 RTT**. - **Persistent, no pipelining:** 2 RTT for the base page (handshake + request) + **1 RTT per object** $\rightarrow$ **2 RTT + n** $\times$ **RTT**. - **Persistent with pipelining:** 2 RTT for base + **1 RTT for all n objects together** $\rightarrow$ **3 RTT**.

**Example (RTT = 100 ms, 10 objects + base):** - Non-persistent: 11 $\times$ 200 = **2200 ms** - Persistent, non-pipelined: 200 + 10 $\times$ 100 = **1200 ms** - Persistent + pipelined: 200 + 100 = **300 ms** (If transmission time $T_t$ per object is given, add it for each object.)

## 14.5 FTP — File Transfer Protocol ★

- Used to **transfer large files**; uses **two TCP connections**:
  - **Control connection → TCP port 21**: used for **commands/responses (control information)**; **only one per session**, stays open for the whole session.
  - **Data connection → TCP port 20**: used for **actual data transfer**; **a new connection is opened for each file**.
- **Out-of-band control** (control info separate from data). FTP is **stateful**.
- **Active mode:** the client opens the control connection; then the **server initiates the data connection** (from port 20) to the client.
- **Passive mode: only the client initiates** both connections (works through firewalls/NAT).

TRAP

Your notes swap the two port numbers ("Control = TCP/20, Data = TCP/21"). The **standard** (and the GATE answer) is **21 = control, 20 = data**.

## 14.6 DHCP — Dynamic Host Configuration Protocol ★

When a computer boots it **doesn't know its IP address, subnet mask, etc.** One way is to enter them manually; **DHCP does it automatically.** - **DHCP provides: IP address, default gateway address, DNS server address, network mask, lease time**, etc. - Uses **UDP**: **server port 67, client port 68**; works through a **relay agent** when the server is on another network.

### DORA process (4 phases) ★

| Step | Message | Details |
|---|---|---|
| **D** | **DHCP Discover** | The client doesn't know any IP → **broadcasts** the message with **source 0.0.0.0, destination 255.255.255.255**, and its **MAC address**. |
| **O** | **DHCP Offer** | **One or more DHCP servers** send an offer with **IP address & other information, plus a lease time**, to the host (as **unicast** using the host's MAC as destination and the server's MAC as source — per the notes; some servers broadcast it). |
| **R** | **DHCP Request** | The host **chooses one offer** and **broadcasts** the accepted offer so that **other DHCP servers know their offer was rejected**. |
| **A** | **DHCP Ack** | The server whose offer was accepted sends **acknowledgement (unicast)** to the host. Host can now use the IP until the **lease** expires (renew at 50 %). |

EXAM SHORTCUT

**Both client-originated messages (Discover, Request) are broadcasts.**

## 14.7 Other protocols — quick facts

| Protocol | What it does | Transport / Port |
|---|---|---|
| **Telnet** | remote terminal (insecure) | TCP 23 |
| **SSH** | secure remote login | TCP 22 |
| **TFTP** | trivial FTP, no auth | **UDP 69** |
| **SNMP** | network management | UDP 161 / 162 |
| **NTP** | time sync | UDP 123 |
| **ICMP** | errors & queries | **IP protocol 1** (no port) |
| **ARP** | IP → MAC | directly inside Ethernet frame (type 0x0806) |

## 14.8 ★★ MASTER TABLE — Protocols, ports, transport (memorise!)

| Application protocol | Port | Transport |
|---|---|---|
| **HTTP** | **80** | TCP |
| **HTTPS** | **443** | TCP |
| **FTP (data / control)** | **20 / 21** | TCP |
| **SSH** | 22 | TCP |
| **Telnet** | 23 | TCP |
| **SMTP** | **25** | TCP |
| **DNS** | **53** | **UDP** (TCP for zone transfer) |
| **DHCP** | **67 (server) / 68 (client)** | **UDP** |
| **TFTP** | 69 | UDP |
| **POP3** | **110** | TCP |
| **IMAP** | **143** | TCP |
| **NTP** | 123 | UDP |
| **SNMP** | 161/162 | UDP |
| **BGP** | **179** | TCP |
| **RIP** | **520** | UDP |
| **OSPF** | – (IP protocol **89**) | IP directly |
| **TCP / UDP / ICMP** in IP header | **6 / 17 / 1** | – |

EXAM SHORTCUT

**Rule of thumb:** *Applications where a lost packet is tolerable or the exchange is tiny and request/response (DNS, DHCP, SNMP, TFTP, RIP, VoIP) → UDP. Applications needing reliable bulk transfer (HTTP, FTP, SMTP, POP, IMAP, SSH, BGP) → TCP.*

CHAPTER

# 15

# Practice Sets with Full Solutions

IN THIS CHAPTER


- Set A — IP addressing & subnetting ( do all)
- Set B — TCP, flow & congestion control ( do all)
- Set C — Fragmentation, ICMP, ARP
- Set D — Routing
- Set E — Application layer
- Set F — MAC, Ethernet, devices
- Set G — Error control & framing
- Set H — Delays


SOLUTION

**How to use:** cover the solution, attempt each question in the time given, then compare. These are **my own questions written in the style of the PYQs** (not copied from past papers). After this, solve the real PYQs of each topic yourself using the question-type → method table (§16.2).

## Set A — IP addressing & subnetting (★★★ do all)

**A1.** Find network address, broadcast address, valid host range and number of hosts for **172.16.37.200/20**.

SOLUTION

Mask 255.255.**240**.0 → block = 256 − 240 = 16 (3rd octet). 37 → multiple of 16 ≤ 37 = **32**.
**Network = 172.16.32.0**, **Broadcast = 172.16.47.255**, hosts = 172.16.32.1 – 172.16.47.254, $2^{12} - 2 = 4094$.

**A2.** Host **192.168.5.77**, mask **255.255.255.224**. Find its network, broadcast, host range.

SOLUTION

/27, block 32. 77 → 64. **Network 192.168.5.64**, **broadcast 192.168.5.95**, hosts **.65 – .94 (30)**.

**A3.** With mask /27, are 192.168.10.100 and 192.168.10.120 in the same subnet?

SOLUTION

100 → block start 96; 120 → block start 96 (96–127). **Same subnet.** ✔

**A4.** Organisation has **130.50.0.0/16**, needs subnets with ≥ **500 hosts each**; maximum number of subnets?

SOLUTION

500 hosts → 9 host bits (510 ≥ 500) → prefix /23 → subnets = $2^{23-16} = 128$; mask **255.255.254.0**.

**A5.** A /24 network is split into **6 equal subnets**. What is the new mask and hosts per subnet?

SOLUTION

6 → need 3 bits ($2^3 = 8 \geq 6$) → **/27 = 255.255.255.224**, **30 hosts** per subnet (8 subnets possible, 2 spare).

**A6.** Aggregate 192.24.0.0/21, 192.24.8.0/21, 192.24.16.0/21, 192.24.24.0/21.

SOLUTION

4 blocks × 2048 = 8192 addresses → prefix 21 − 2 = **/19** → **192.24.0.0/19** (0 divisible by 32 ✔).

**A7.** Can 200.1.1.0/24 and 200.1.2.0/24 be aggregated into one /23?

SOLUTION

**No.** A /23 must start at an even third octet (...0 or ...2); 200.1.1.0 isn't aligned.

**A8.** IP header has HLEN = 7 and Total length = 600. How many data bytes?

SOLUTION

Header = $7 \times 4 = 28$ B $\rightarrow$ data = $600 - 28 =$ **572 B**.

**A9.** `0.0.0.0` and `255.255.255.255` — differences?

SOLUTION

0.0.0.0 = "this host / unknown source (DHCP) / default route"; 255.255.255.255 = **limited broadcast** on the local network (never forwarded by a router).

**A10.** Divide **192.168.1.0/24** into subnets for 100, 50, 25, 10 hosts and one point-to-point link.

SOLUTION

See §7.5: /25, /26, /27, /28 and /30 at 0, 128, 192, 224, 240.

**A11 (forwarding with LPM).**

| Prefix | | Iface |
|---|---|---|
| `150.20.0.0/16` | $\rightarrow$ | **I1** |
| `150.20.64.0/18` | $\rightarrow$ | **I2** |
| `150.20.64.0/20` | $\rightarrow$ | **I3** |
| `default` | $\rightarrow$ | **I4** |

| Destination | Working | Answer |
|---|---|---|
| 150.20.70.5 | /16 ✔, /18: 70&192 = 64 ✔, /20: 70&240 = 64 ✔ $\rightarrow$ longest /20 | **I3** |
| 150.20.90.1 | /18 ✔ (90&192 = 64), /20: 90&240 = 80 $\neq$ 64 ✘ | **I2** |
| 150.20.130.9 | /18: 130&192 = 128 $\neq$ 64 ✘; /16 ✔ | **I1** |
| 151.1.1.1 | none | **I4** |

## Set B — TCP, flow & congestion control (★★★ do all)

**B1.** Client ISN = 1000, server ISN = 5000; the third handshake segment carries 200 B. Give seq/ack of all three segments and the server's next ack.

SOLUTION

SYN: seq 1000 • SYN+ACK: seq 5000, ack 1001 • ACK+data: seq 1001, ack 5001 (200 B) • server acks **1201**.

**B2.** A client sends 3 segments of 500 B each after the handshake, with client ISN = 7000. What is the ack number after the second segment?

SOLUTION

SYN used 7000, so data begins at 7001. Seg1: 7001–7500; Seg2: 7501–8000 $\rightarrow$ **ack = 8001**.

**B3.** A connection sends exactly 1000 bytes of data from the client. How many sequence numbers does the client use (including SYN and FIN)?

SOLUTION

SYN (1) + data (1000) + FIN (1) = **1002**.

**B4.** TCP header HLEN field = 8; IP header HLEN = 5; IP total length = 500 B. Application data?

SOLUTION

TCP header = 32 B, IP header = 20 B $\rightarrow$ data = $500 - 20 - 32$ = **448 B**.

**B5.** ssthresh = 8, cwnd = 1, no loss. cwnd after 6 RTTs?

SOLUTION

1, 2, 4, 8, 9, 10, **11**. $\rightarrow$ **11 MSS**.

**B6.** Tahoe; timeout occurs when cwnd = 24. cwnd after 5 RTTs?

SOLUTION

ssthresh = 12, cwnd = 1 $\rightarrow$ 2, 4, 8, 12, **13**. $\rightarrow$ **13**.

**B7.** Reno; 3 dup-ACKs when cwnd = 24; cwnd after 3 RTTs?

SOLUTION

ssthresh = 12, cwnd = 12 $\rightarrow$ 13, 14, **15**.

**B8.** Initial ssthresh = 32. Timeouts occur when cwnd = 20 and later when cwnd = 12 (Tahoe). Give the cwnd sequence from the first timeout until just after the second timeout's recovery reaches the threshold.

SOLUTION

After the first timeout: ssthresh = 10, cwnd = 1 $\rightarrow$ 2, 4, 8, 10 (=ssthresh) $\rightarrow$ 11, 12 (timeout) $\rightarrow$ ssthresh = 6, cwnd = 1 $\rightarrow$ 2, 4, 6 $\rightarrow$ then CA 7, 8 ...

**B9.** Window = 64 KB, RTT = 100 ms, link = 100 Mbps. TCP throughput and utilisation?

SOLUTION

65536 × 8 / 0.1 = **5.24 Mbps** → utilisation **5.24 %**.

**B10.** Link 100 Mbps, RTT 50 ms. Minimum window to keep the link full?

SOLUTION

BDP = $10^8 \times 0.05 = 5 \times 10^6$ bits = **625,000 B ≈ 610 KB**.

**B11.** How long before the 32-bit TCP sequence numbers wrap on a 10 Gbps link?

SOLUTION

$2^{32} \times 8 \,/\, 10^{10}$ = **3.44 s**.

**B12.** Which flags are set in the three handshake segments?

SOLUTION

**SYN** | **SYN+ACK** | **ACK**. (Termination: **FIN** | **FIN+ACK** or **ACK** | **ACK**.)

**B13 (state).** Which side enters TIME_WAIT, and for how long?

SOLUTION

The **active closer (usually the client)**; **2 × MSL** (e.g. MSL = 60 s → 120 s).

**B14 (sliding window).** 1 Mbps link, frames 1000 bits, one-way $T_p$ = 270 ms (geostationary satellite). (a) SW efficiency? (b) window to fill the pipe? (c) sequence bits for GBN/SR?

SOLUTION

$T_t$ = 1 ms, a = 270. (a) $\eta = 1/(1+540) = 1/541 \approx$ **0.185 %**. (b) N = **541**. (c) GBN: $542 \le 2^m \rightarrow$ **m = 10**. SR: $1082 \le 2^m \rightarrow$ **m = 11**.

**B15 (loss).** 10 packets, window 5, packet 3 lost once, all other transmissions fine; sender sends the whole window before the timeout. Count transmissions in GBN and SR.

SOLUTION

**GBN:** 0–4 (5) + 5,6,7 sent as acks arrive (3) + retransmit 3–7 (5) + 8,9 (2) = **15**. **SR:** 10 + 1 = **11**.

## Set C — Fragmentation, ICMP, ARP

**C1.** Datagram data = 4000 B, header 20 B, MTU 1500. Give number of fragments, offsets and MF.

SOLUTION

D = 1480 → fragments 1480, 1480, 1040 → offsets **0, 185, 370**, MF **1, 1, 0**; total lengths 1500, 1500, 1060.

**C2.** MTU = 1000, datagram total length 5000 (header 20). Number of fragments and the size of the last?

SOLUTION

Data 4980; D = $\lfloor 980/8 \rfloor \times 8 = 976$ → $\lceil 4980/976 \rceil$ = **6** fragments; last = $4980 - 5 \times 976$ = **100 B** data (120 B total).

**C3.** A fragment: offset 100, MF = 0, total length 200, header 20. Original data length?

SOLUTION

data = 180; starts at 800 → **980 B**.

**C4.** Which fields change at every router? **TTL & header checksum.** Which *may* change at a router that fragments? Total length, MF, offset (and options/HLEN).

**C5.** Router receives a packet with TTL = 1. What happens?

SOLUTION

It decrements to 0, **discards** the packet and sends **ICMP Time Exceeded** to the source.

**C6.** Does ICMP generate an error for a datagram that carries an ICMP error message? For a non-first fragment?

SOLUTION

**No** in both cases.

**C7.** To send to a host in a different network, whose MAC does a host ARP for?

SOLUTION

The MAC of the **default gateway / next-hop router**.

## Set D — Routing

**D1 (DV).** Links P–Q 4, Q–R 2, P–R 9. Final tables?

SOLUTION

After one exchange: P→R = min(9, 4+2) = **6 via Q**; R→P = **6 via Q**; Q: P4, R2.

**D2 (Dijkstra from A).** Edges A–B 4, A–C 2, B–C 1, B–D 5, C–D 8, C–E 10, D–E 2.

SOLUTION

A0 → C2 → B = min(4, 2+1) = **3** → D = min(3+5, 2+8) = **8** → E = min(2+10, 8+2) = **10** (path A-C-B-D-E).

**D3.** RIP: what number represents infinity, and why?

SOLUTION

**16.** Limits count-to-infinity; max usable path = 15 hops.

**D4.** Which protocol/AS-level? RIP = DV, intra-AS (UDP 520); OSPF = LS, intra-AS (IP 89); BGP = path vector, inter-AS (TCP 179).

## Set E — Application layer

**E1.** Page with 1 base + 10 objects, RTT = 100 ms. Load time in (a) non-persistent, (b) persistent non-pipelined, (c) persistent pipelined (ignore transmission times)?

SOLUTION

(a) 11 × 2 RTT = **2200 ms**; (b) 2 RTT + 10 RTT = **1200 ms**; (c) 2 RTT + 1 RTT = **300 ms**.

**E2.** Which protocols between: (i) sender UA → its mail server, (ii) mail server → mail server, (iii) mail server → receiver UA?

SOLUTION

(i) **SMTP** (ii) **SMTP** (iii) **POP3 / IMAP / HTTP**.

**E3.** Which DHCP messages are broadcast?

SOLUTION

**Discover** and **Request** (client side). Offer & Ack are typically unicast.

**E4.** True/False: FTP uses a single TCP connection. → **False** (control 21 + data 20).

**E5.** DNS uses which transport normally? → **UDP port 53**; TCP for zone transfers.

**E6.** Number of DNS messages in the iterative example (client → local → root → TLD → auth → auth)? → **10**.

## Set F — MAC, Ethernet, devices

**F1.** 10 Mbps CSMA/CD, cable 2 km, v = $2\times10^8$ m/s. Minimum frame size?

SOLUTION

$T_p$ = 10 µs; L = 2 × 10 µs × $10^7$ = **200 bits**.

**F2.** Gigabit Ethernet with a 512-bit minimum frame. Maximum cable length?

SOLUTION

$T_t$ = 512 ns ≥ $2T_p$ → $T_p$ ≤ 256 ns → **51.2 m**.

**F3.** Pure ALOHA at G = 1 and slotted ALOHA at G = 1?

SOLUTION

Pure: $e^{-2}$ = **0.135**; slotted: $e^{-1}$ = **0.368**.

**F4.** Two stations collide; first race both pick from {0,1}. Probability of a second collision? → **1/2**. After A wins and sends a new frame (K=1) while B is at K = 2: P(A wins) = **5/8**, P(B wins) = **1/8**, P(collision) = **1/4**.

**F5.** A switch has 8 ports, all used by individual hosts. Collision domains and broadcast domains? → **8** and **1**. If the 8-port device is a **hub** → **1** and **1**.

## Set G — Error control & framing

**G1.** CRC with g(x) = $x^3$ + x + 1, message 1101 → transmitted word? → **1101001**.

**G2.** Code {00000, 01011, 10101, 11110}: $d_{\min}$ and capability?

SOLUTION

Pairwise distances 3 or 4 → $d_{\min}$ **= 3** → **detects 2, corrects 1**.

**G3.** Check bits needed for 11 data bits in a Hamming code? → **4** ($2^4 = 16 \geq 11 + 4 + 1$).

**G4.** Which g(x) detects all odd-bit errors? (a) $x^4 + x^3 + x + 1$ (b) $x^4 + x^2 + 1$

SOLUTION

(a) has 4 (even) terms → g(1) = 0 → has factor (x + 1) ✔. (b) has 3 terms → g(1) = 1 → ✘.

**G5.** Bit-stuff the data `0111111011111 0` (flag 01111110). → see worked example in §3.2: result **0111110101111100**.

**G6.** Which protocol layer detects a corrupted frame, and which detects a corrupted segment? → DLL (CRC in trailer); transport (TCP/UDP checksum). IP checks **only its header**.

## Set H — Delays

**H1.** 10 packets of 1000 bits through two store-and-forward links of 1 Mbps ($T_p$ = 0). Total time? → (h + n − 1) × $T_t$ = (2 + 9) × 1 ms = **11 ms**.

**H2.** Message 8000 bits on a 2 Mbps link, d = 3000 km, v = $2\times10^8$ m/s. Total delay? → $T_t$ = 4 ms; $T_p$ = 15 ms; **19 ms**.

CHAPTER

# 16

# Formula Sheet & 30-Minute Revision

IN THIS CHAPTER


- 16.1 One-page formulas
- 16.2 Question type → method
- 16.3 Top exam traps
- 16.4 30-minute last-day revision plan


## 16.1 One-page formulas

| Topic | Formula |
|---|---|
| Transmission time | $T_t = \frac{L}{B}$ |
| Propagation time | $T_p = \frac{d}{v}$ |
| RTT | $\mathbf{2}T_p$ $(+\ T_t\text{'s})$ |
| BDP | $B \times \text{RTT}$ |
| a | $\frac{T_p}{T_t}$ |

| Topic | Formula |
|---|---|
| Stop-and-wait efficiency | $\frac{1}{1+2a}$ |
| Throughput | $\eta \times B$ (SW) ; $N\,\eta_{SW}\,B$ (sliding) |
| Window to fill the pipe | $1+2a$ |
| Seq bits: GBN | $N \le 2^m - 1$ |
| Seq bits: SR | $N \le 2^{m-1}$ |
| General | $N_S + N_R \le 2^m$ |
| Expected transmissions (loss p) | $\frac{1}{1-p}$ |
| CSMA/CD min frame | $L_{\text{min}} = 2\,T_p\,B$ |
| Backoff after K-th collision | wait slot $\times\ \mathcal{U}\{0, \dots, 2^K - 1\}$ |
| Pure / Slotted ALOHA | $G\,e^{-2G}$ (max 0.184) / $G\,e^{-G}$ (max 0.368) |
| Hamming bound | $2^r \ge m + r + 1$ |
| Detect / correct | $d_{\text{min}} \ge s+1$ / $2t+1$ |
| CRC bits | **degree of g(x)** |
| Hosts in /n | $2^{32-n} - 2$ |
| Block size | $256 -$ **mask octet** |
| Network / Broadcast | **IP & mask / net \| ~mask** |
| Aggregate k blocks | prefix $-\ \log_2$**k**, aligned to total size |
| IP header bytes | HLEN $\times\ 4$ (20–60) ; data = TL $-$ header |
| Fragment data size | $\left\lfloor \frac{\text{MTU} - \text{hdr}}{8} \right\rfloor \times 8$ |

| Topic | Formula |
|---|---|
| Fragment offset | $\frac{\text{first byte}}{8}$ |
| MSS (Ethernet, TCP/IP) | **1500 − 20 − 20 = 1460** |
| TCP seq of next segment | **seq + bytes** (SYN/FIN count 1) |
| Ack number | **next byte expected** |
| rwnd | buffer − (received − read) |
| Send window | min(rwnd, cwnd) |
| Slow start | **doubles per RTT** until ssthresh |
| CA | **+1 MSS per RTT** |
| On loss | ssthresh = $\frac{\text{cwnd}}{2}$; Tahoe/timeout: cwnd = 1; Reno 3-dup: cwnd = ssthresh |
| Window-limited throughput | $\frac{W}{\text{RTT}}$ |
| Wrap-around time | $\frac{2^n \text{ bytes}}{\text{rate}}$ |
| RTT estimate | $R_s = \alpha R_m + (1-\alpha)R_s$, $\alpha = 1/8$ |
| HTTP page loads | non-persistent **2RTT/obj**; persistent **RTT/obj**; pipelined **1 RTT total** (+ 2 RTT for base) |

## 16.2 Question type → method

| If the question says … | Do this |
|---|---|
| "network address / broadcast / hosts for IP/n" | **Block-size trick** (§7.3) |
| "divide into k subnets / each with h hosts" | borrow bits / **VLSM largest-first** (§7.4–7.5) |
| "which interface / next hop" | **Longest-prefix match** (§8.2) |
| "aggregate / summarise routes" | **contiguity + alignment + power of 2** (§8.3) |
| "fragments / offset / MTU" | $\left\lfloor \frac{\text{MTU} - 20}{8} \right\rfloor \times 8$, offset = start/8 (§9.3) |
| "fields changed by router" | **TTL + checksum** |
| "cwnd after k RTTs" | **table: SS doubling, cap at ssthresh, CA +1** (§13.5) |
| "seq / ack numbers in handshake" | **SYN, FIN consume 1; pure ACK none** (§12.1) |
| "throughput of TCP / window" | $\frac{\min(\text{rwnd}, \text{cwnd})}{\text{RTT}}$ (§12.7) |
| "channel utilisation (SW/SR/GBN)" | **a =** $T_p/T_t$**, N/(1+2a)** (§5) |
| "min frame size / max cable length" | $L = 2\,T_p\,B$ (§6.4) |
| "probability CSMA/CD backoff" | enumerate pairs from $\{0 \ldots 2^K - 1\}$ (§6.5) |
| "DV update / count to infinity" | **min(cost + neighbour's distance)**, table per round (§10) |
| "which protocol/port/layer" | **Master table** (§14.8) |
| "CRC property" | factor conditions (§4.6) |

## 16.3 Top exam traps

1. **"Usable hosts" =** $2^h - 2$**.** Subtract network & broadcast.
2. **MTU vs MSS**; fragment data multiples of **8**.
3. **LPM beats order:** most specific prefix wins, not the first listed.
4. **Slow start is capped by ssthresh**; ssthresh is set from the cwnd **at the time of loss**.
5. **ACK number = next expected byte**, not the last received.
6. **SYN and FIN consume a sequence number; pure ACKs don't.**
7. **Bits vs bytes; K =** $10^3$ **for rates,** $2^{10}$ **for sizes.**
8. **ARP gives the next hop's MAC; IPs stay constant end-to-end while MACs change every hop.**
9. **IP header checksum covers only the header;** TCP/UDP checksum covers data too.
10. **FTP: control 21, data 20. DHCP uses UDP 67/68. BGP uses TCP.**
11. In GBN **receiver window = 1**, out-of-order frames are **discarded**.
12. **SW/ SR/ GBN window limits:** $1$, $2^{m-1}$, $2^m - 1$.

## 16.4 30-minute last-day revision plan

| Minutes | Revise |
|---|---|
| 0–8 | Subnetting block trick, VLSM steps, LPM, aggregation (Ch 7–8) |
| 8–16 | TCP: handshake numbers, flags, ack rules, congestion table, window-limited throughput (Ch 11–13) |
| 16–21 | Fragmentation recipe + header fields + ARP/ICMP facts (Ch 9) |
| 21–25 | DV tables, count-to-infinity, RIP/OSPF/BGP (Ch 10) |
| 25–28 | Port table, DHCP, DNS, HTTP, SMTP/POP/IMAP (Ch 14) |
| 28–30 | CSMA/CD $L_{\min}$, backoff probability, SW efficiency |

CHAPTER

# 17

# Appendices

IN THIS CHAPTER

- Appendix A — Where each item of your lecture notes is covered
- Appendix B — Corrections to the handwritten notes (so you don't lose marks)
- Appendix C — How to attempt PYQs after this book

## Appendix A — Where each item of your lecture notes is covered

| Lecture-notes topic | Chapter |
|---|---|
| TCP/IP 5-layer stack, IP address parts, IPv4 32 bits, rigid addressing | 1, 7.1 |
| Classful addressing & ranges | 7.2 |
| Classless addressing; 12.24.34.127/21; 220.8.24.255/25; 167.199.170.82/27 | 7.3 |
| DBA, limited broadcast (255.255.255.255), 0.0.0.0 | 7.2 |
| Subnetting: 192.168.9.0/24 into 4 and $\geq$ 10 hosts; first/last subnet confusion | 7.4 |
| VLSM: 14.24.74.0/24 (10,60,120); 172.16.0.0/16 (500,200,100,62,22) | 7.5 |

| Lecture-notes topic | Chapter |
|---|---|
| Forwarding of IP packets, routing table fields, LPM, example with 201.4.22.128 | 8.2 |
| Supernetting (140.24.7.0/24 example & R2 table, 140.24.7.193) | 8.3 |
| Network layering, protocol, TCP/IP stack, hosts vs routers | 1 |
| Bandwidth, throughput example (2 Mbps), latency, propagation, transmission, queuing, link vs channel | 2 |
| Data link layer functions, sublayers (broadcast vs point-to-point) | 3.1 |
| Multiple access protocols, random access, the four MAC questions | 6.1 |
| CSMA/CD, jamming, no ACK, $L_{\min} \geq 2{\cdot}T_p{\cdot}$BW | 6.4 |
| Backoff algorithm, collision probabilities | 6.5 |
| Error control: dataword/codeword (k=4,r=2), Hamming distance, min HD for detection, linear block code, parity | 4.1–4.5 |
| Cyclic code, CRC (1001/1011 $\rightarrow$ 1001110), polynomial form, analysing CRC, single-bit, two isolated errors, odd errors | 4.6 |
| Flow control, Stop-and-Wait cases, efficiency, Earth–Moon example, throughput, avg transmissions (0.2 error $\rightarrow$ 125) | 5.1–5.2 |
| Bandwidth-delay product | 2.4 |
| Sliding window, GBN (cases C1–C4, seq/window), SR (cases C1–C6, seq/window), $N_S{+}N_R \leq 2^m$, efficiency, throughput | 5.3–5.6 |
| Framing, byte stuffing ($ / \ example), bit stuffing | 3.2 |
| Routing protocols, DV routing, examples, count-to-infinity, link state (completed) | 10 |
| IP header (all fields, HLEN, TTL, protocol, checksum, options) | 9.1 |
| Fragmentation, MTU/MSS, offset, examples (5000 B / MTU 800; MTU 576 / 1484 B) | 9.2–9.3 |
| ICMP, error & query messages, rules | 9.4 |
| ARP, packet format, 4 use-cases | 9.5 |
| Transport layer, ports, UDP header | 11.1–11.2 |
| TCP, seq/ack numbers, windows, header, flags, window-scale, checksum | 11.3–11.5 |
| TCP connection establishment (table 5000/8000/5001/8001) | 12.1 |
| Connection termination (3-way, 4-way), TCP states | 12.2–12.3 |
| Acknowledgement policy flowchart | 12.4 |
| Retransmission (timeout, 3 dup ACKs, fast retransmit) | 12.5 |
| Flow control in TCP (rwnd example) | 12.7 |
| Congestion control, slow start, delayed ACK, CA, ssthresh, Tahoe, Impl. 2, worked example | 13 |
| TCP timers, smoothed RTT | 12.5–12.6 |
| E-mail (SMTP, POP3, IMAP) | 14.2 |
| DNS (hierarchy, servers, iterative/recursive) | 14.3 |
| FTP, HTTP (persistent/non-persistent/pipelined) | 14.5, 14.4 |

| Lecture-notes topic | Chapter |
|---|---|
| DHCP (DORA) | 14.6 |

NOTE

Topics in your PYQ analysis marked □ (Tier 4: CRC/Hamming, error detection) are kept short but complete. Topics **cut** as unnecessary: history of networks, detailed physical-layer signalling, security, detailed Wi-Fi/Bluetooth.

## Appendix B — Corrections to the handwritten notes (so you don't lose marks)

| # | In the notes | Standard / exam-correct version |
|---|---|---|
| 1 | FTP: control = TCP/20, data = TCP/21 | **Control = 21, Data = 20** |
| 2 | TIME_WAIT "2 MSS" | **2** × **MSL** (Maximum Segment Lifetime) |
| 3 | Backoff range **[0,** $2^{K-1}$**]** (B gets {0,1,2} at K=2) | **{0 ...** $2^K - 1$**}** → {0,1,2,3}; P(A wins) = **5/8** |
| 4 | CRC polynomial example labelled dataword $x^4 + x^3$ | The working corresponds to dataword $x^3$ **+ 1** (1001); remainder $x^2$ + x, codeword $x^6$ **+** $x^3$ **+** $x^2$ **+ x** |
| 5 | ARP request: target H/W address "all 1s" | Target H/W field = **all 0s** in the request; the **frame** destination is the broadcast FF:FF:FF:FF:FF:FF (all 1s) |
| 6 | Class E leftmost bits written as 11110 (if so in your copy) | Class E begins **1111** (240–255) |
| 7 | Bit stuffing: "0 followed by five 1s" | Standard: **after any five consecutive 1s insert a 0** |
| 8 | GBN window "$< 2^m$" | Same thing: $N \leq 2^m - 1$ |
| 9 | "Taho TCP" | **TCP Tahoe**; "Implementation 2" ≈ **Reno** |
| 10 | "13 root servers each with 250 TLDs" | 13 root server *names* (A–M), replicated worldwide; ~250+ TLDs are stored at root, not "per root server" |

## Appendix C — How to attempt PYQs after this book

1. Do **Tier 1** clusters first: subnetting/CIDR, TCP/UDP/congestion — all years 2018–2026.
2. Then **Tier 2**: application layer (protocol roles), routing (DV, count-to-infinity), then **fragmentation/ARP/ICMP**.
3. For each wrong answer, write **which formula/trick** from §16.2 you missed.
4. Re-solve the same questions after 7 days without looking.
5. In the exam spend $\leq$ **1 minute per 1-mark** and $\leq$ **2–3 minutes per 2-mark** CN question; subnetting and congestion questions should take **under 90 seconds** once the tricks are automatic.

SHORTCUT **Final advice:** CN gives $\approx$ 8 marks and is the most *formula-driven, predictable* subject of GATE CSE. Master **Chapters 7, 8, 11, 12, 13 and the port table** and you secure $\approx$ 5–6 of those marks with near certainty.

*End of book.*